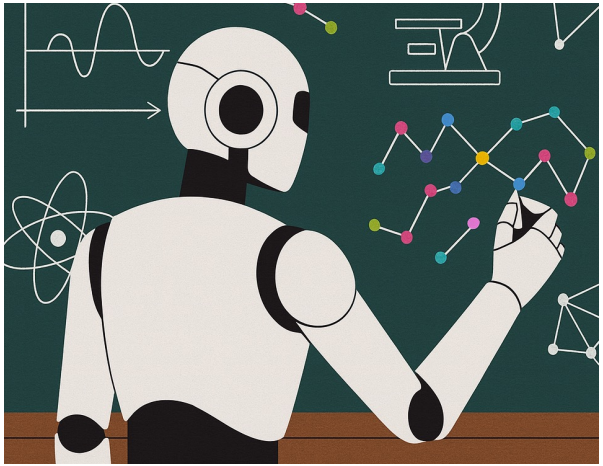
AI for Quantum Physics



Winter semester 2025/26

Today's topic:

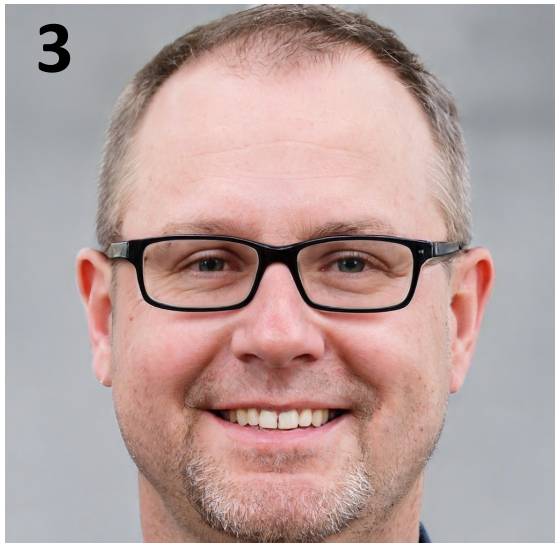
- Autoencoders and Variational autoencoders
- Generative adversarial networks (GANs)
- Applications

If you have any questions, feel free to reach out!
xuemei.gu@uni-jena.de

Which face is real?



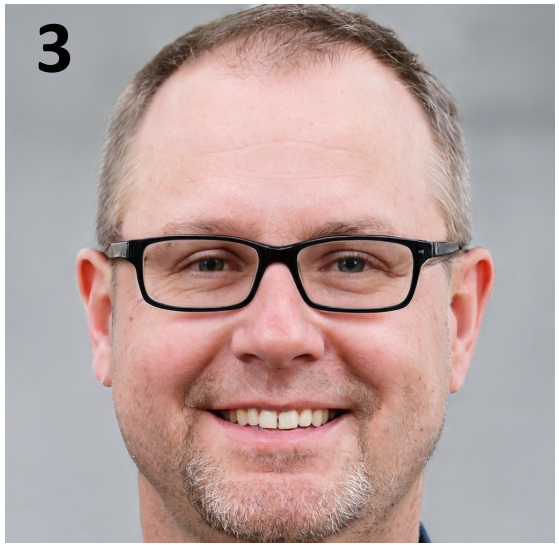




Which face is real?







https://thispersondoesnotexist.com

Supervised vs unsupervised learning

Supervised Learning

Data: (x, y)

x is data, y is label

Learn function to map $x \rightarrow y$



Dog

Classification

Examples:

Classification, regression, object detection, semantic segmentation, etc

Supervised vs unsupervised learning

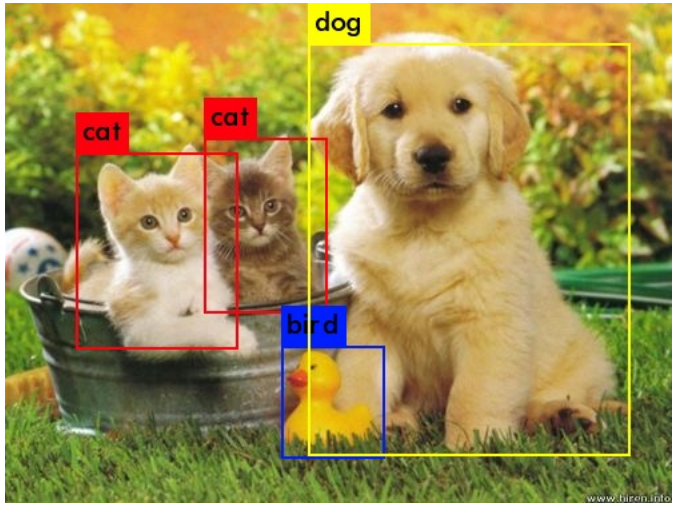
Supervised Learning

Data: (x, y)

x is data, y is label

Learn function to map $x \rightarrow y$

Examples:
Classification, regression, object detection, semantic segmentation, etc



Object detection

Supervised vs unsupervised learning

Supervised Learning

Data: (x, y)
x is data, y is label

Learn function to map $x \rightarrow y$

Examples:
Classification, regression, object detection, semantic segmentation, etc



A cat sitting on a suitcase on the floor

Image captioning

Supervised vs unsupervised learning

Unsupervised Learning

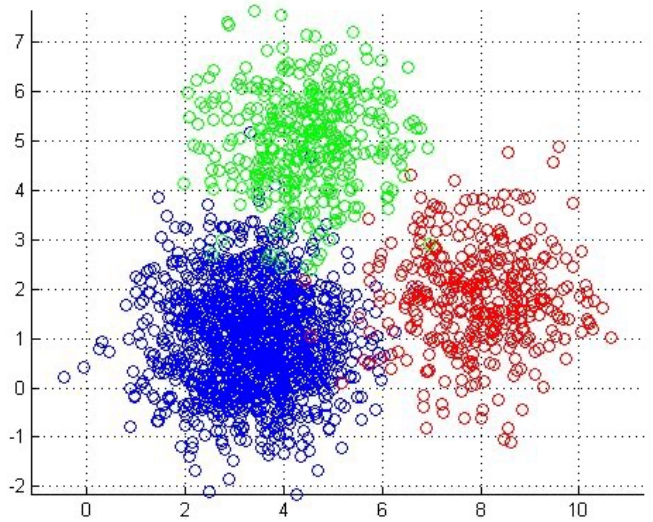
Data: x

x is data, **no label**

Learn some hidden or underlying structure of the data

Examples:

Clustering, feature or dimensionality reduction, etc



Clustering

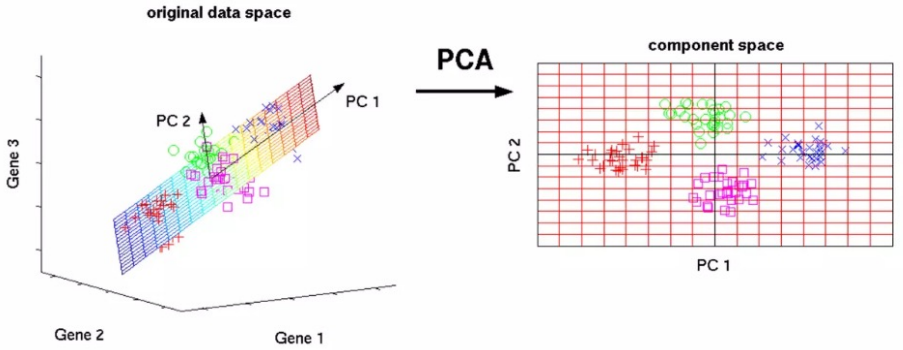
Supervised vs unsupervised learning

Unsupervised Learning

Data: x

x is data, **no label**

Learn some hidden or underlying structure of the data



Examples:
Clustering, feature or dimensionality reduction, etc

Principal component analysis (PCA)
dimensionality reduction

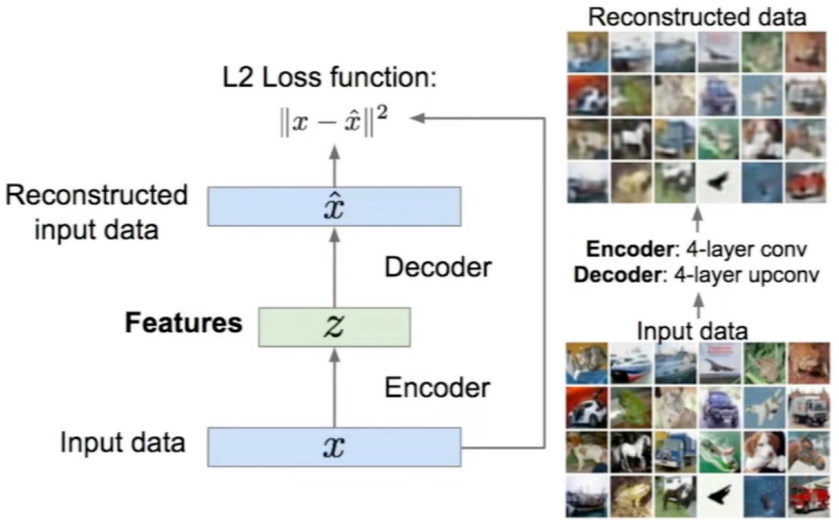
Supervised vs unsupervised learning

Unsupervised Learning

Data: x

x is data, **no label**

Learn some hidden or underlying structure of the data



Examples:

Clustering, feature or dimensionality reduction, etc

Autoencoder

Feature Learning

Statistical Generative Models



+

Model family, loss function, optimization algorithm, etc.

Data

Learning

Prior Knowledge

Image x



A probability distribution p(x)

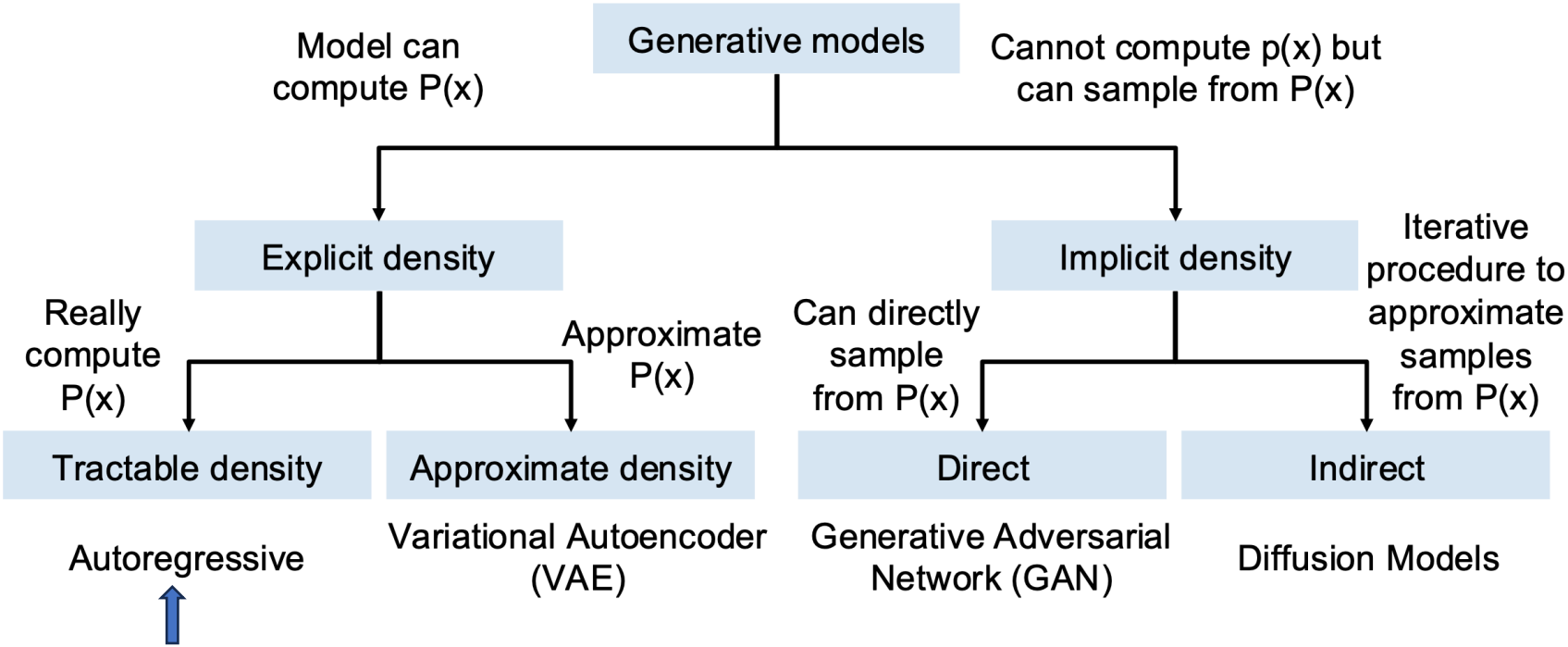
probability p(x)

Sampling from p(x) **generates** new images:



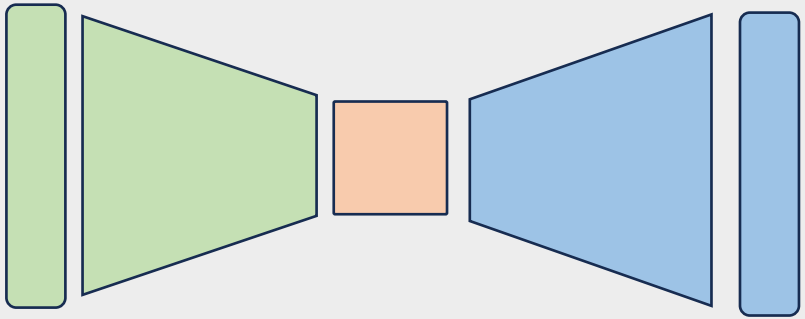
Grover and Ermon, DGM Tutorial

Generative Model

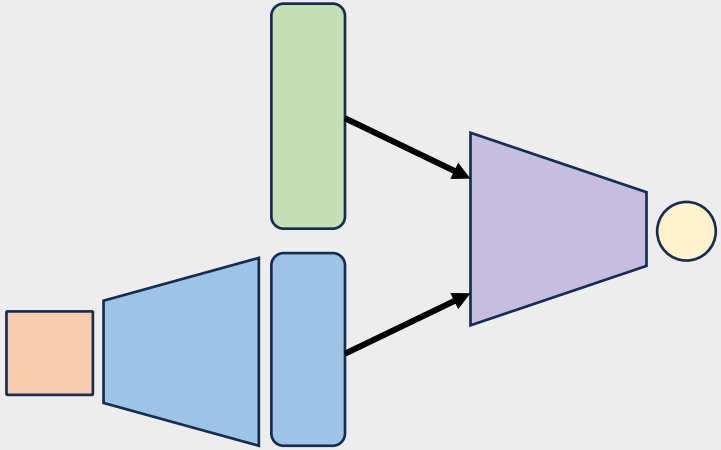


Latent variable models

Autoencoder and Variational Autoencoders (VAEs)

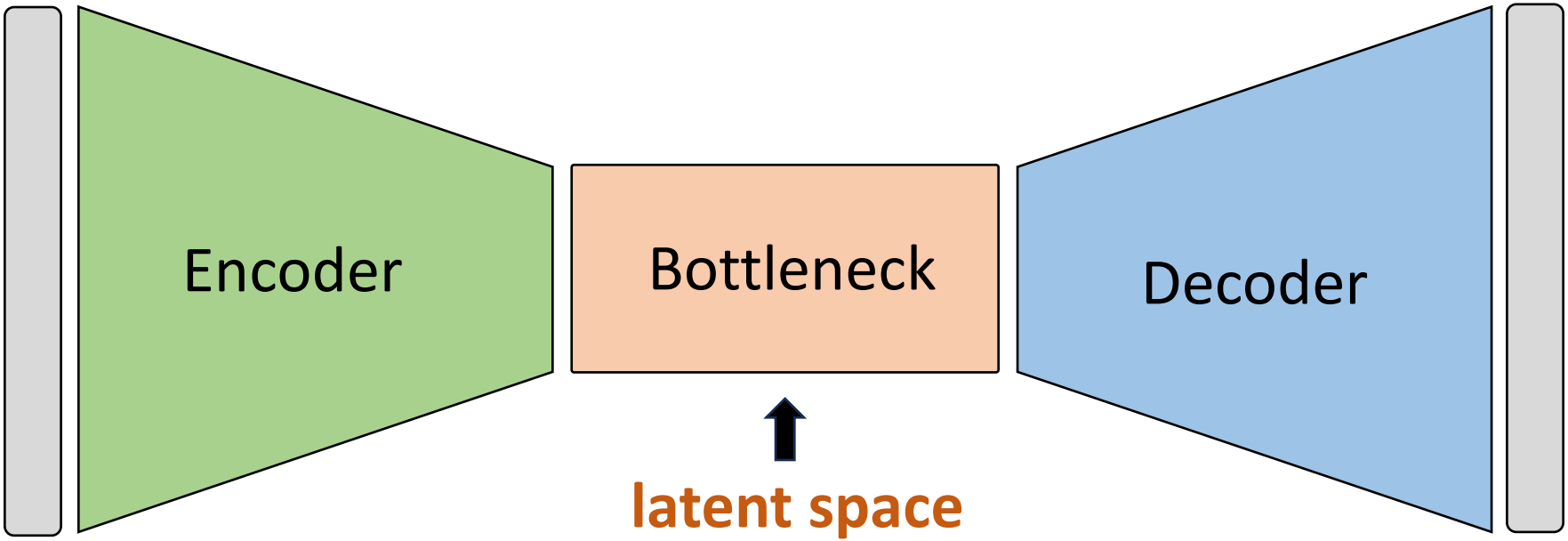


Generative Adversarial Networks (GANs)



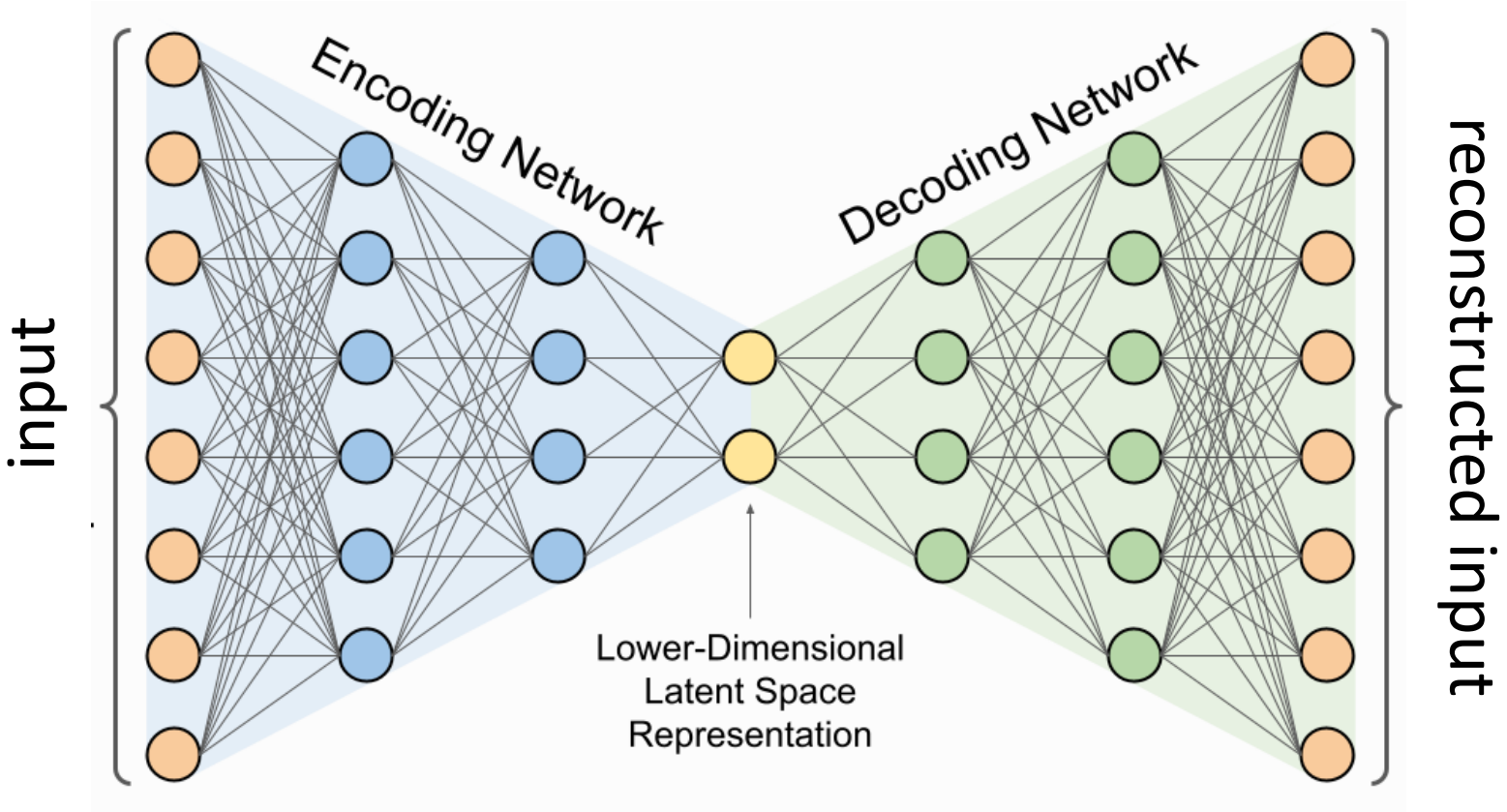
Autoencoder

Unsupervised approach for learning a lower-dimensional feature representation from **unlabeled training data**

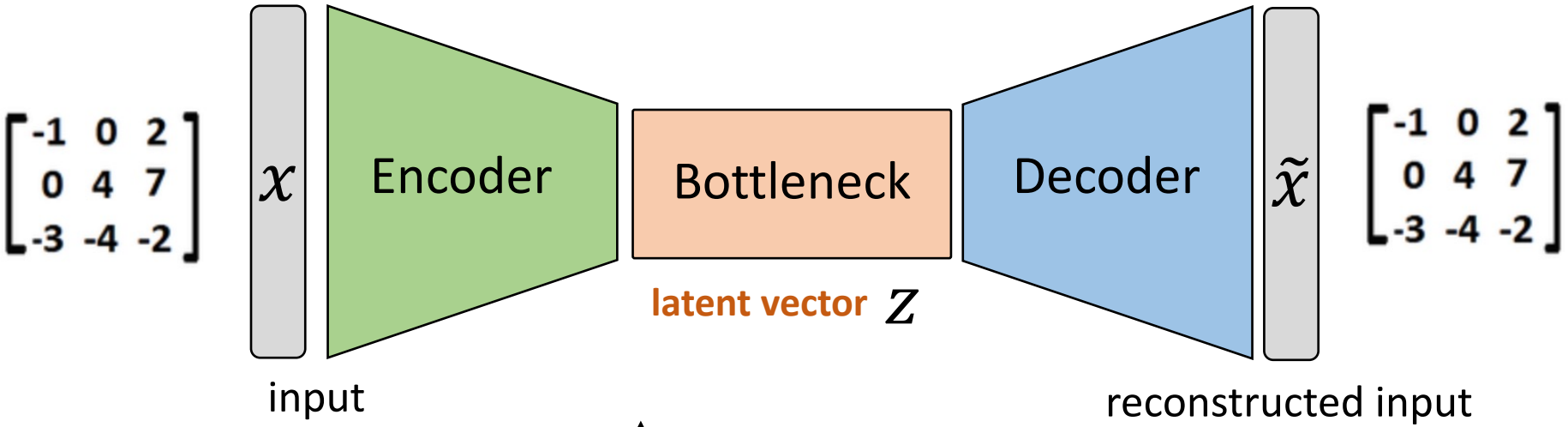


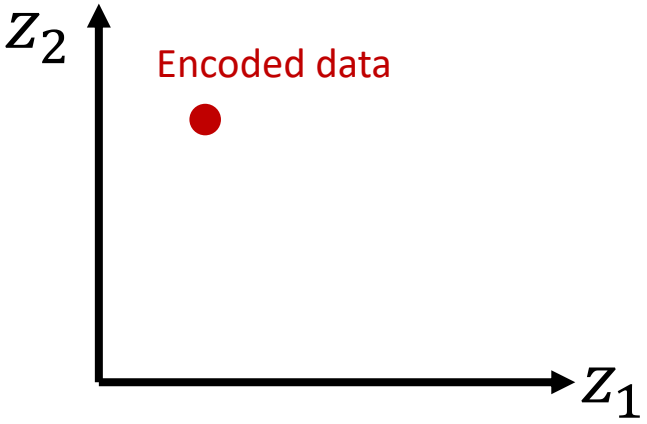
A low-dimensional space that captures the essential features

Autoencoder

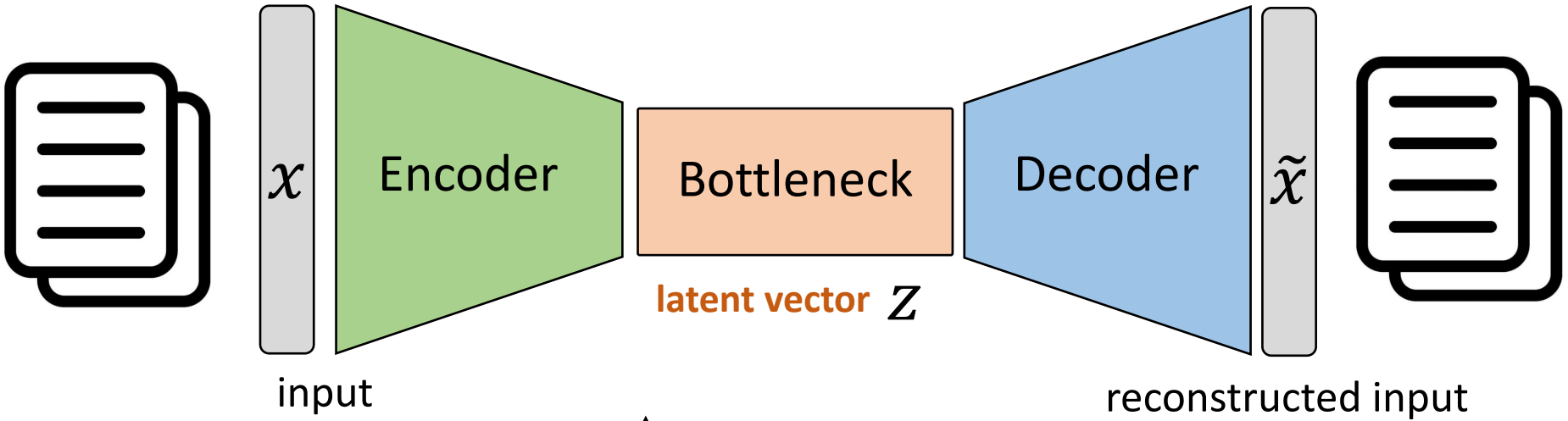


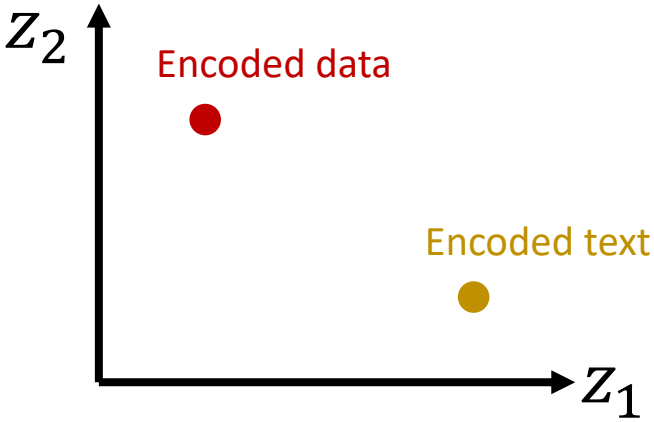
Autoencoder



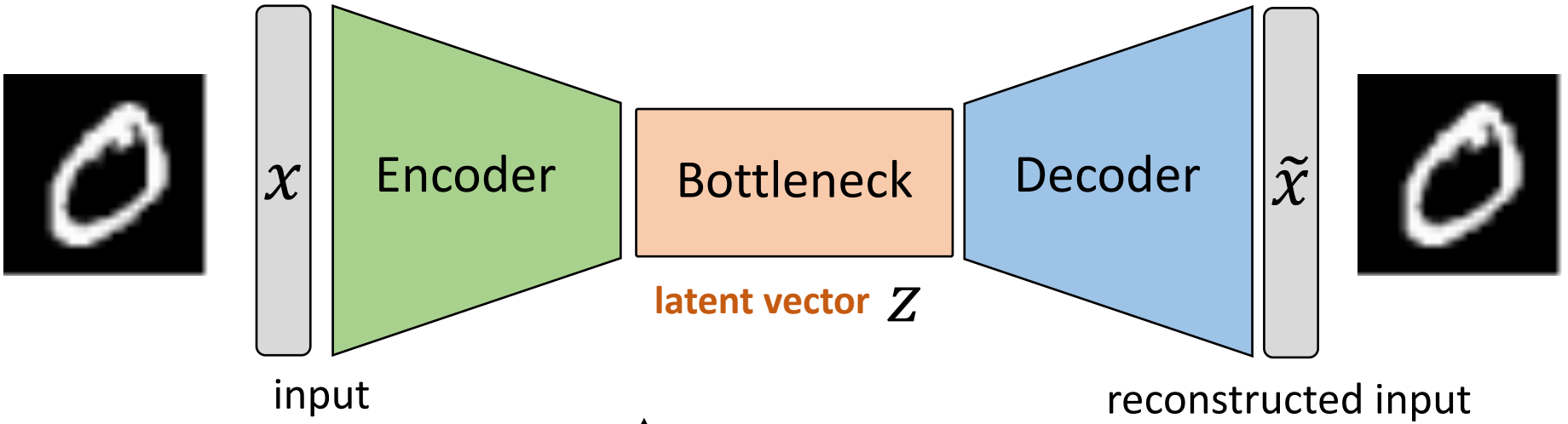


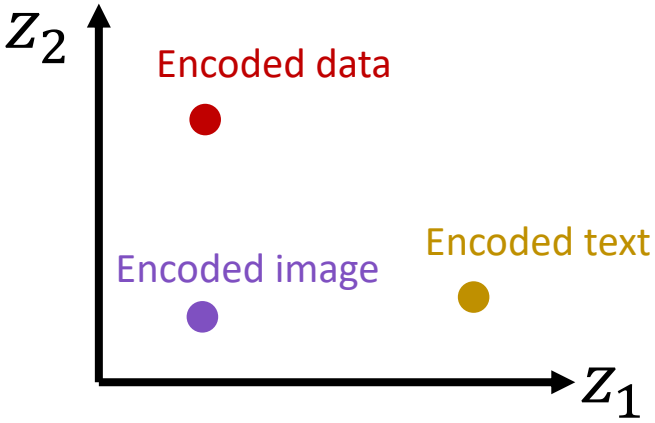
Autoencoder



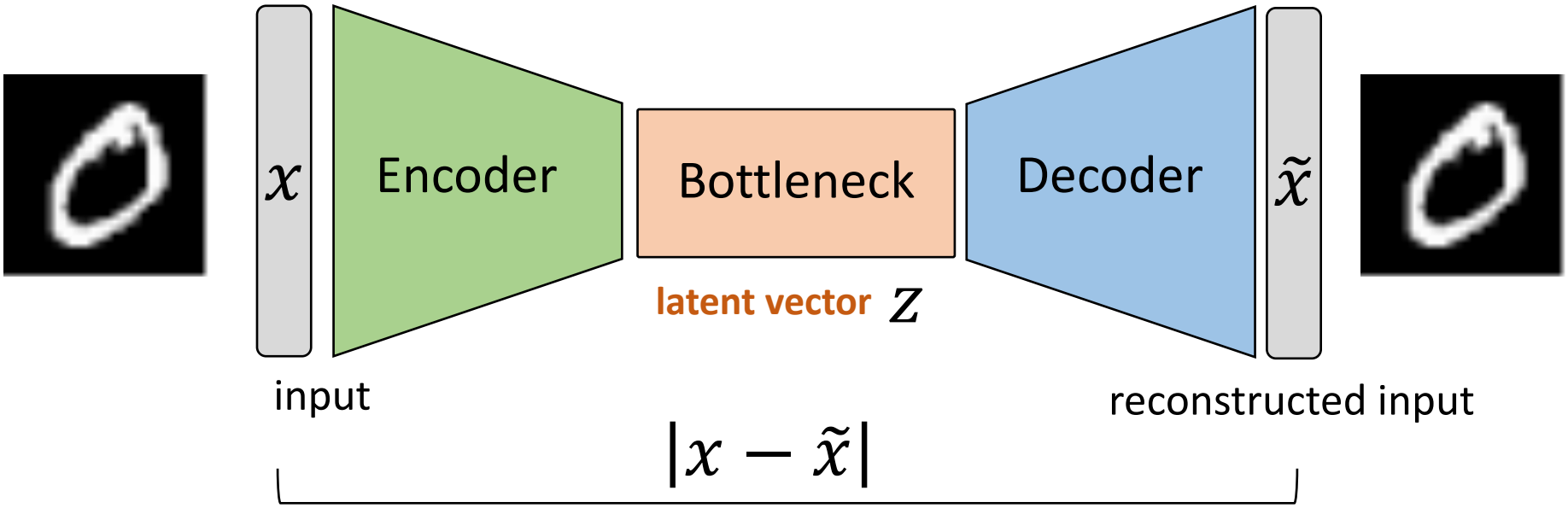


Autoencoder



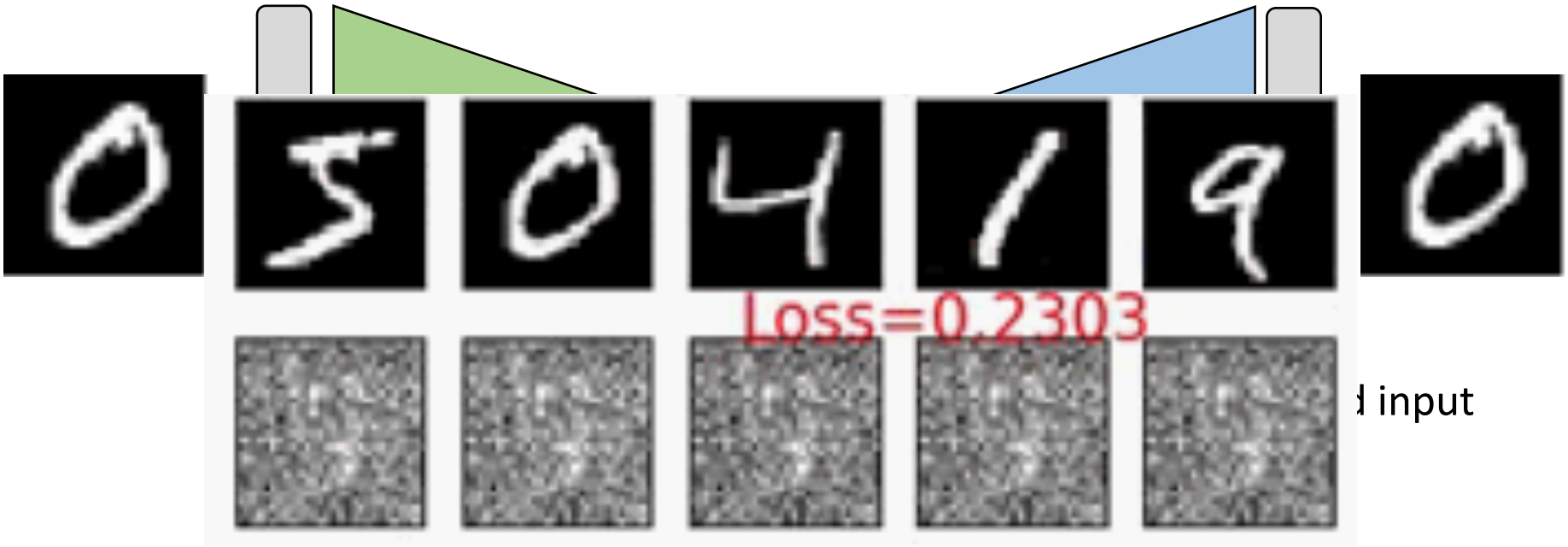


Train Autoencoder



Mean squared error $\mathcal{L}(x, \tilde{x}) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \tilde{x}_i)^2$

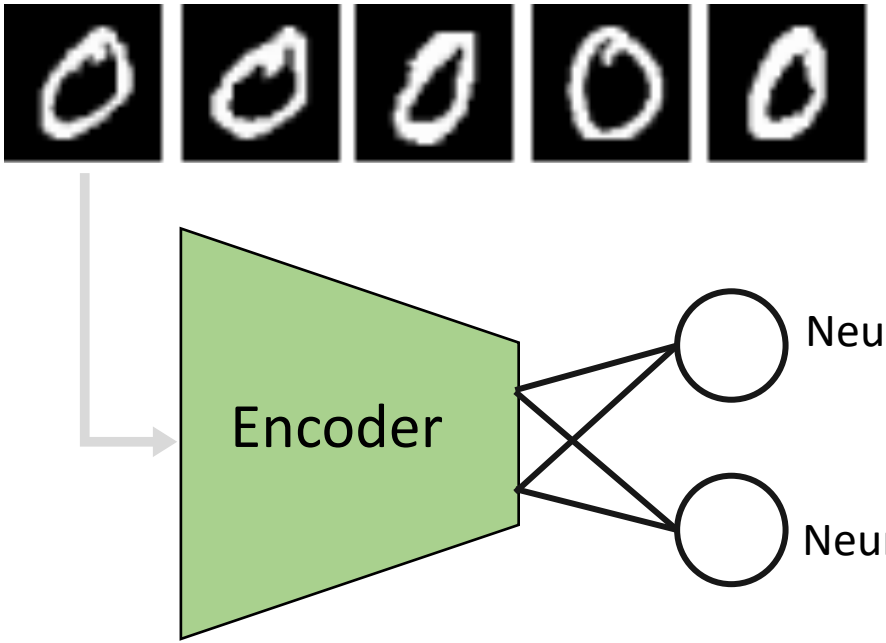
Train Autoencoder

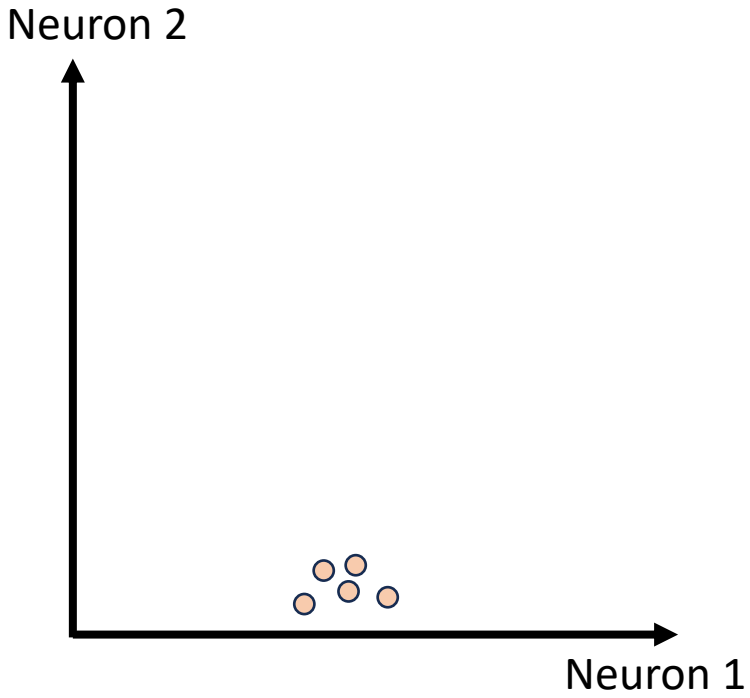


l input

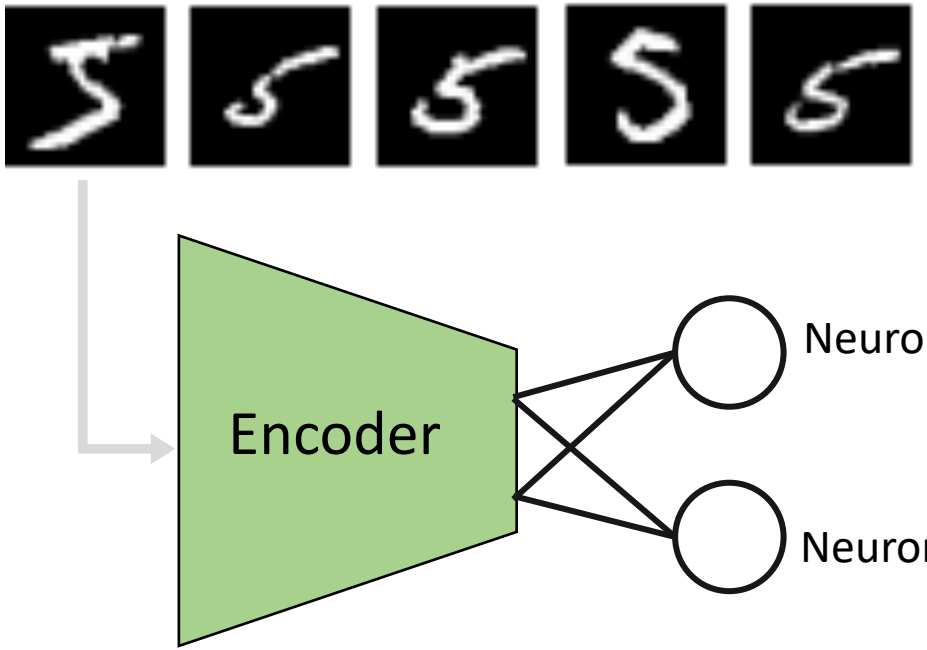
Mean squared error $$\mathcal{L}(x, \tilde{x}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \tilde{x}_i)^2$$

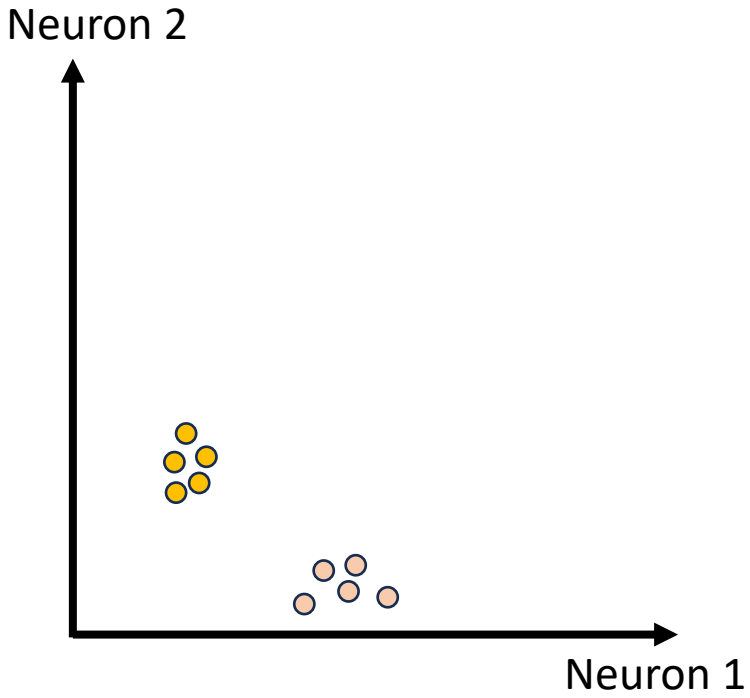
Autoencoder



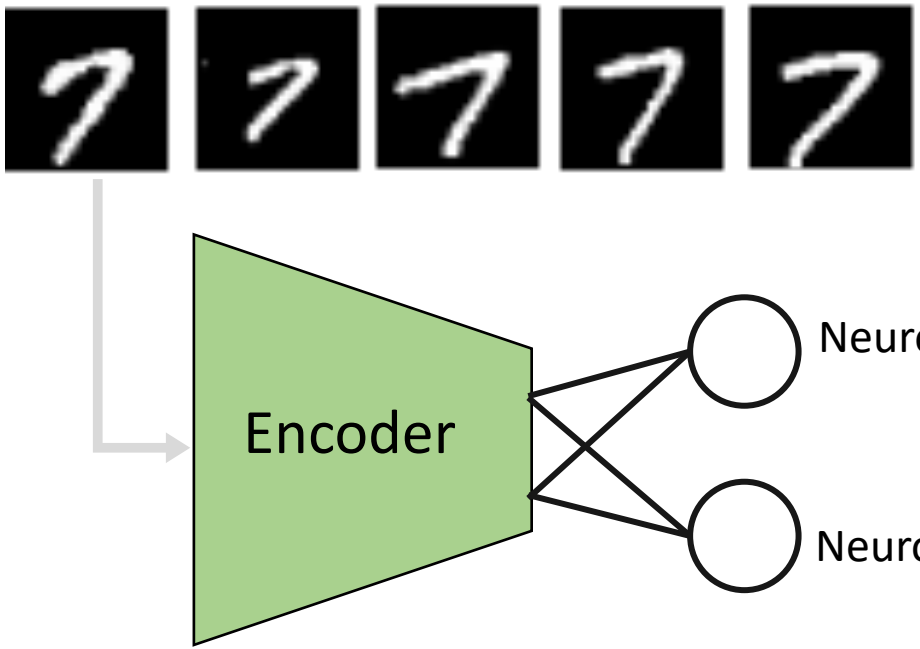


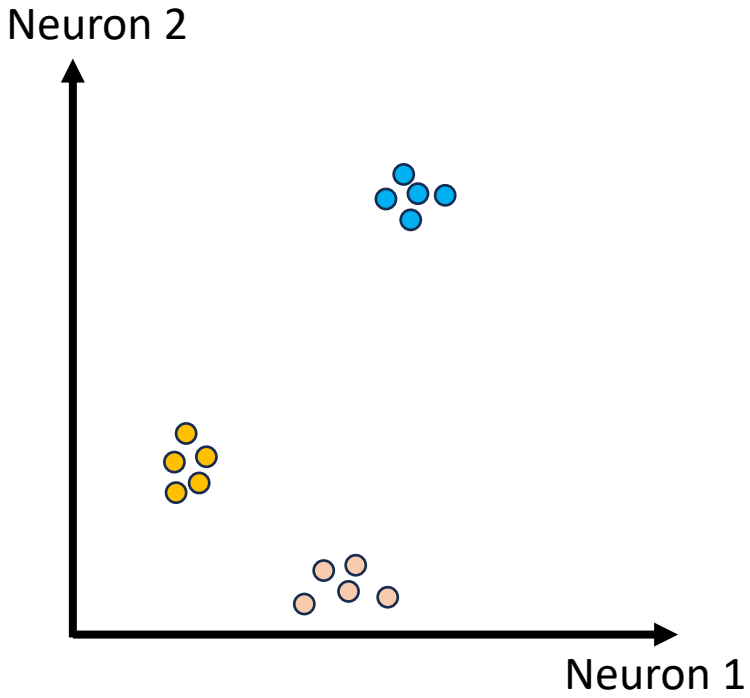
Autoencoder



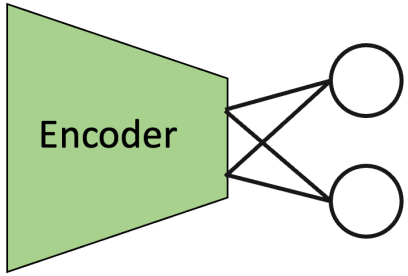


Autoencoder



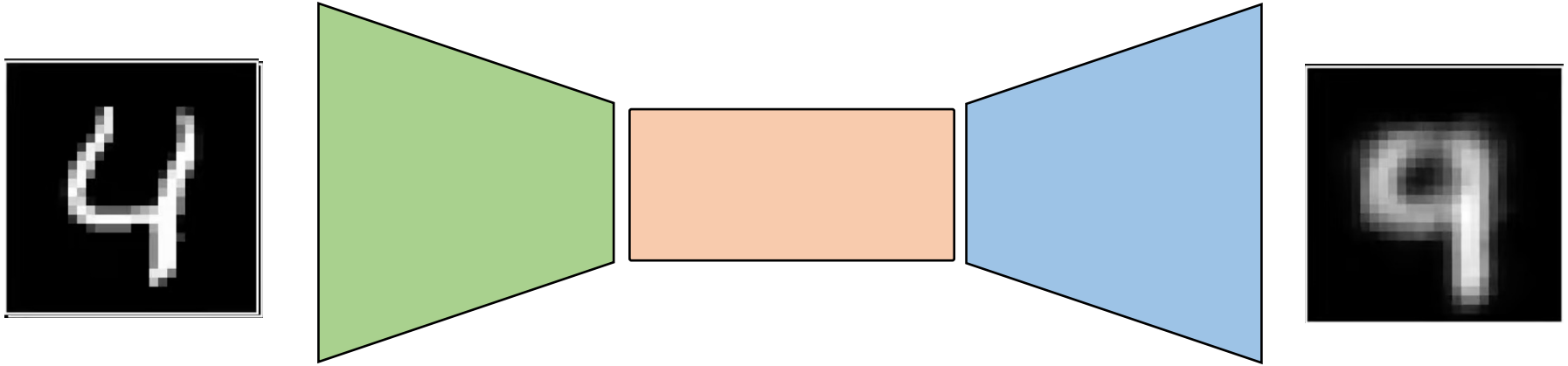


Autoencoder



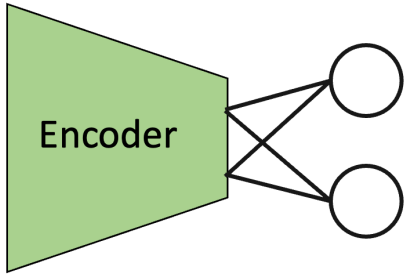
Neuron =2, 3, 4,….

Latent dimension



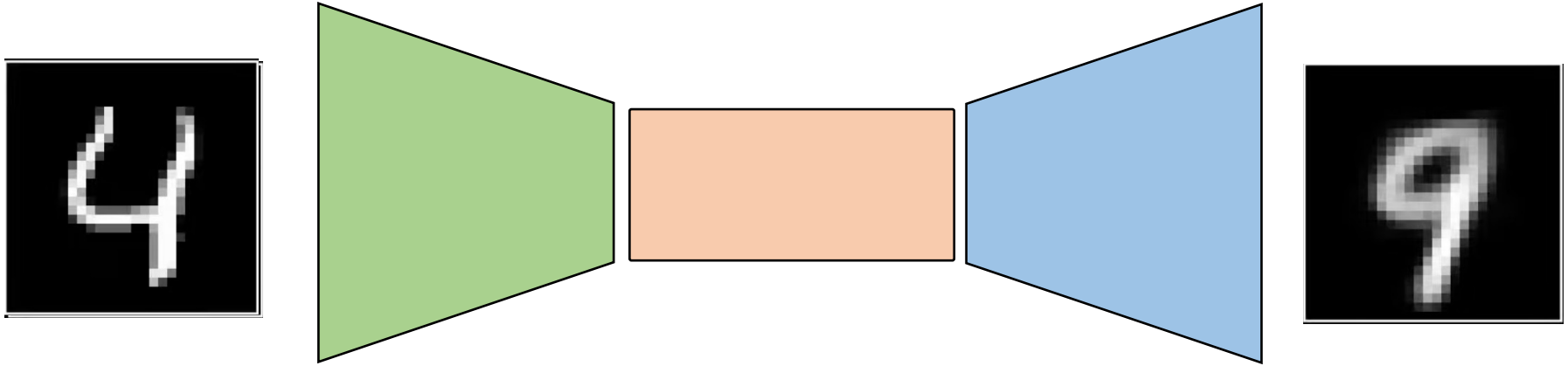
Latent dimension =1

Autoencoder



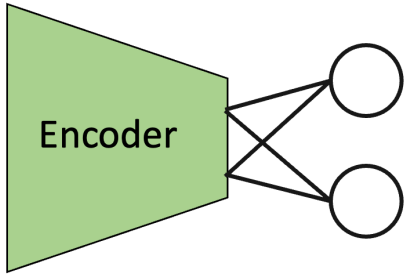
Neuron =2, 3, 4,….

Latent dimension



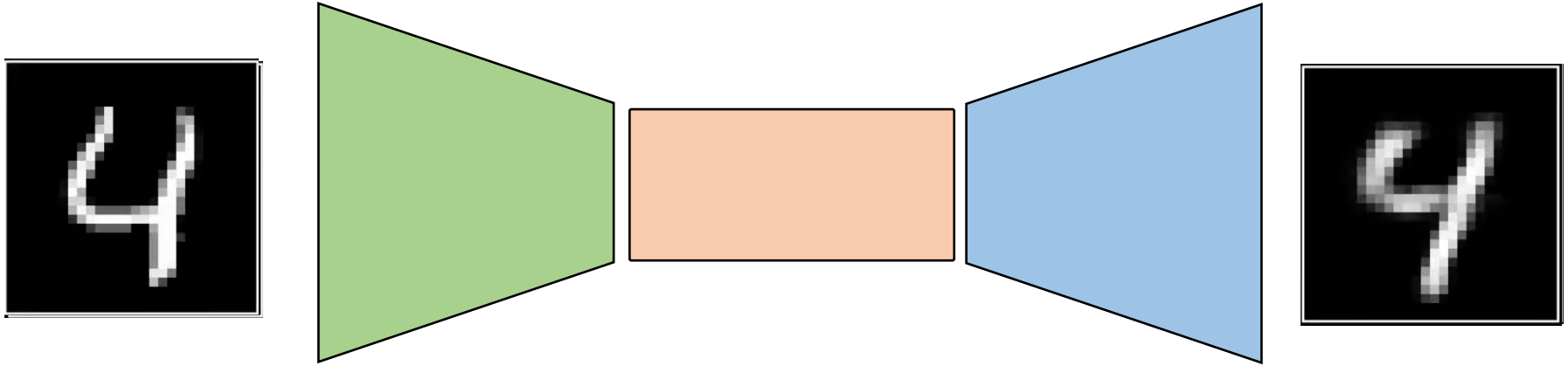
Latent dimension =2

Autoencoder



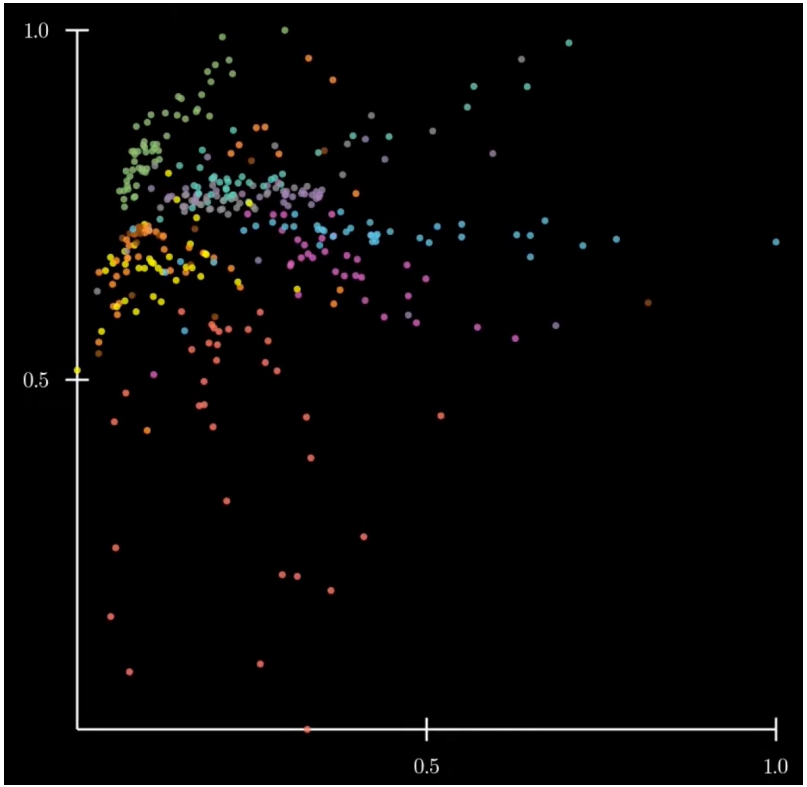
Neuron =2, 3, 4,….

Latent dimension

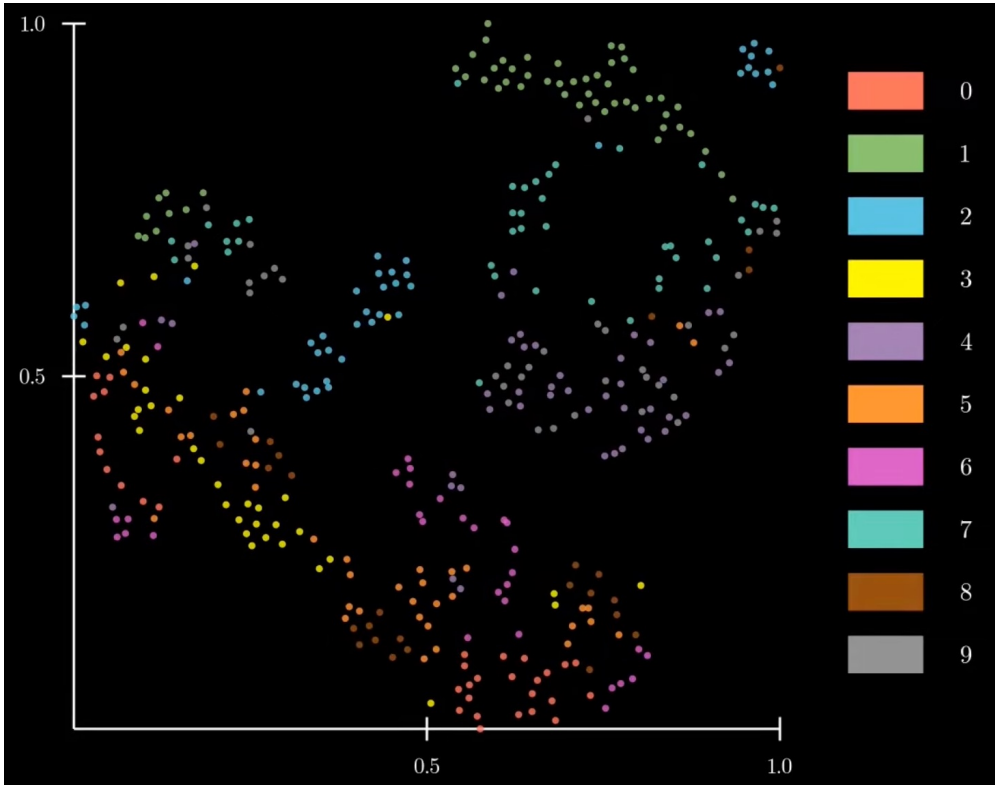


Latent dimension =9

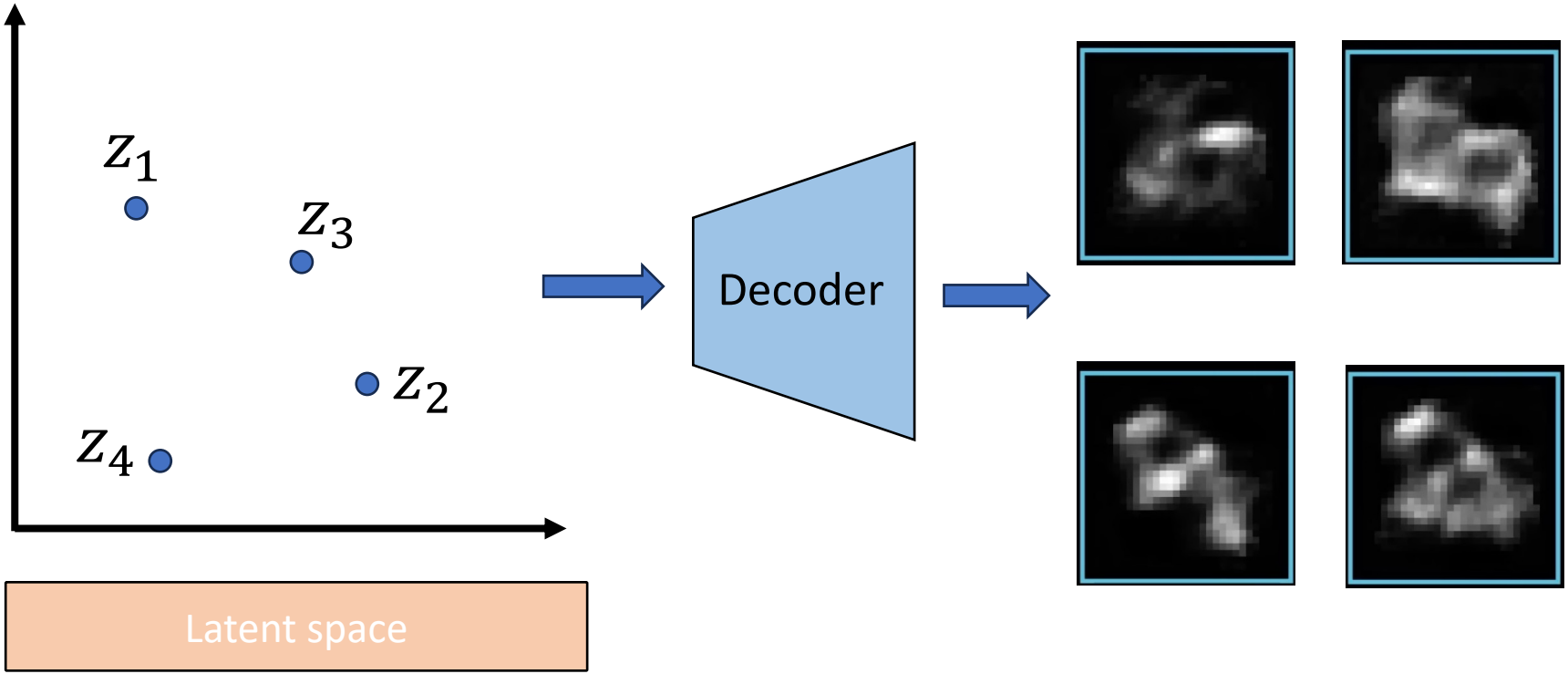
Latent dimension =2



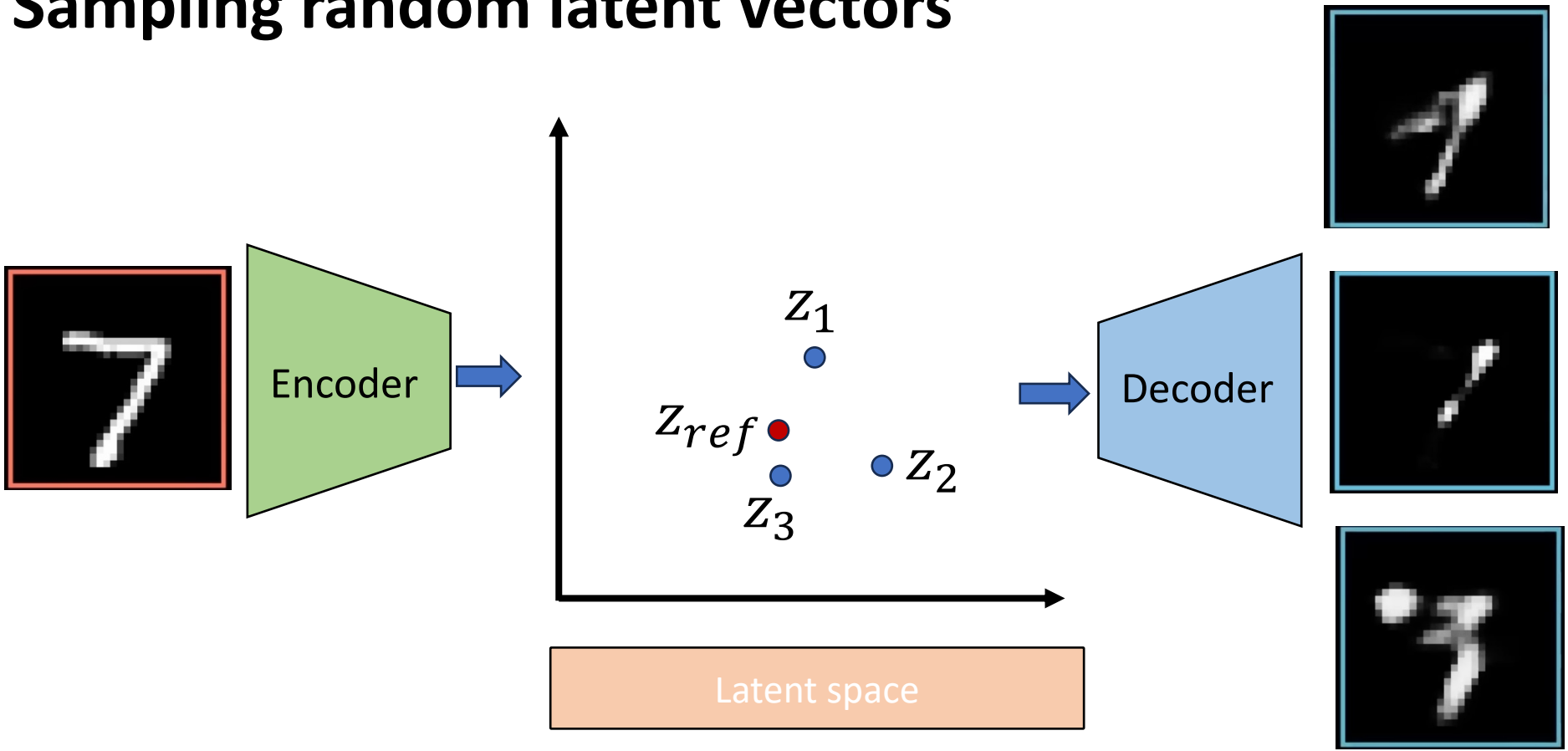
Latent dimension =5



Sampling random latent vectors



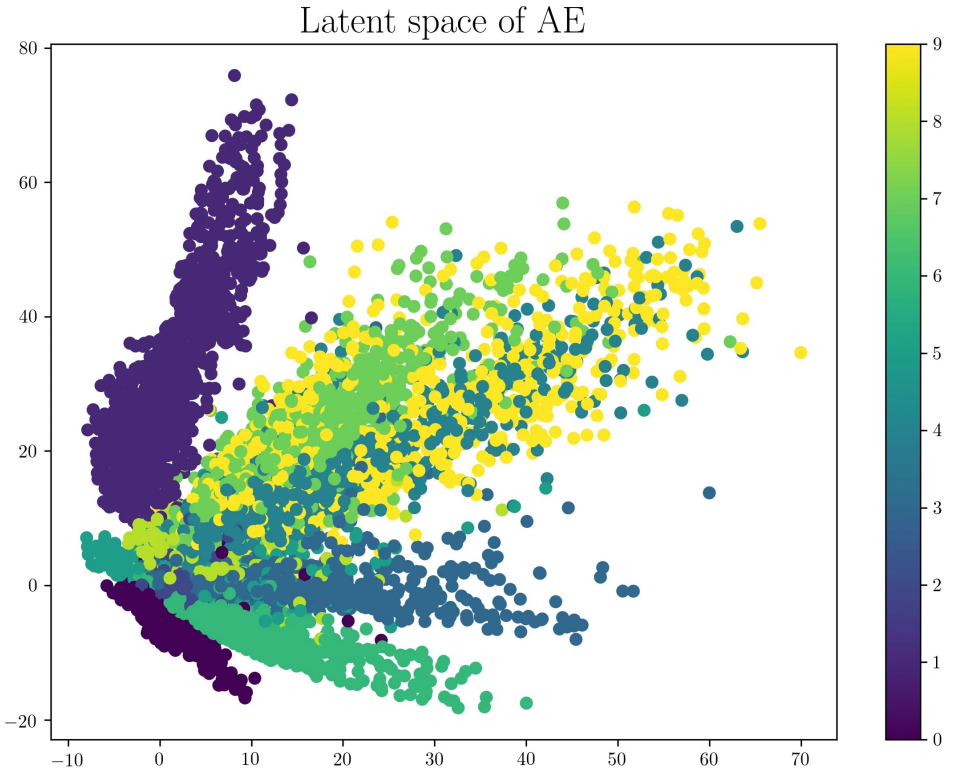
Sampling random latent vectors



Problems with Autoencoders

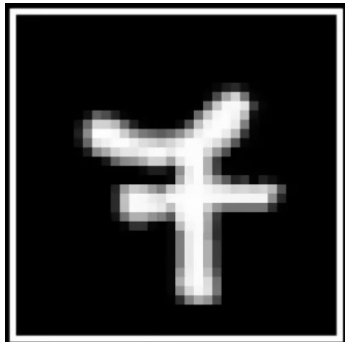
- Gaps in the latent space
- Separability in the latent space
- Discrete latent space

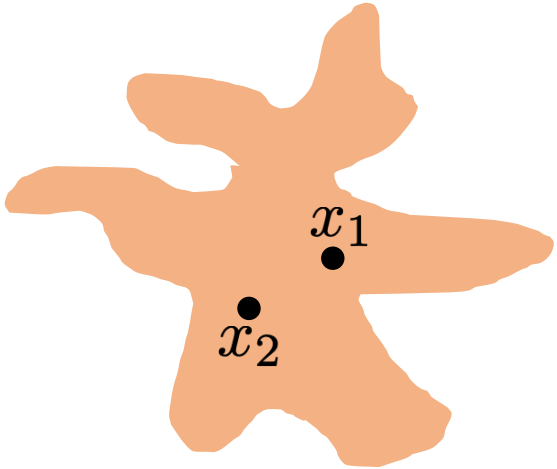
learn a low dimensional representation of the input, but hard to work with as a generative model

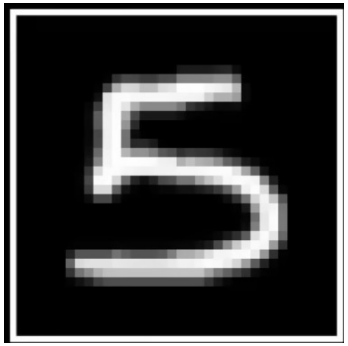


Organize the latent space!

Variational Autoencoders (VAEs)

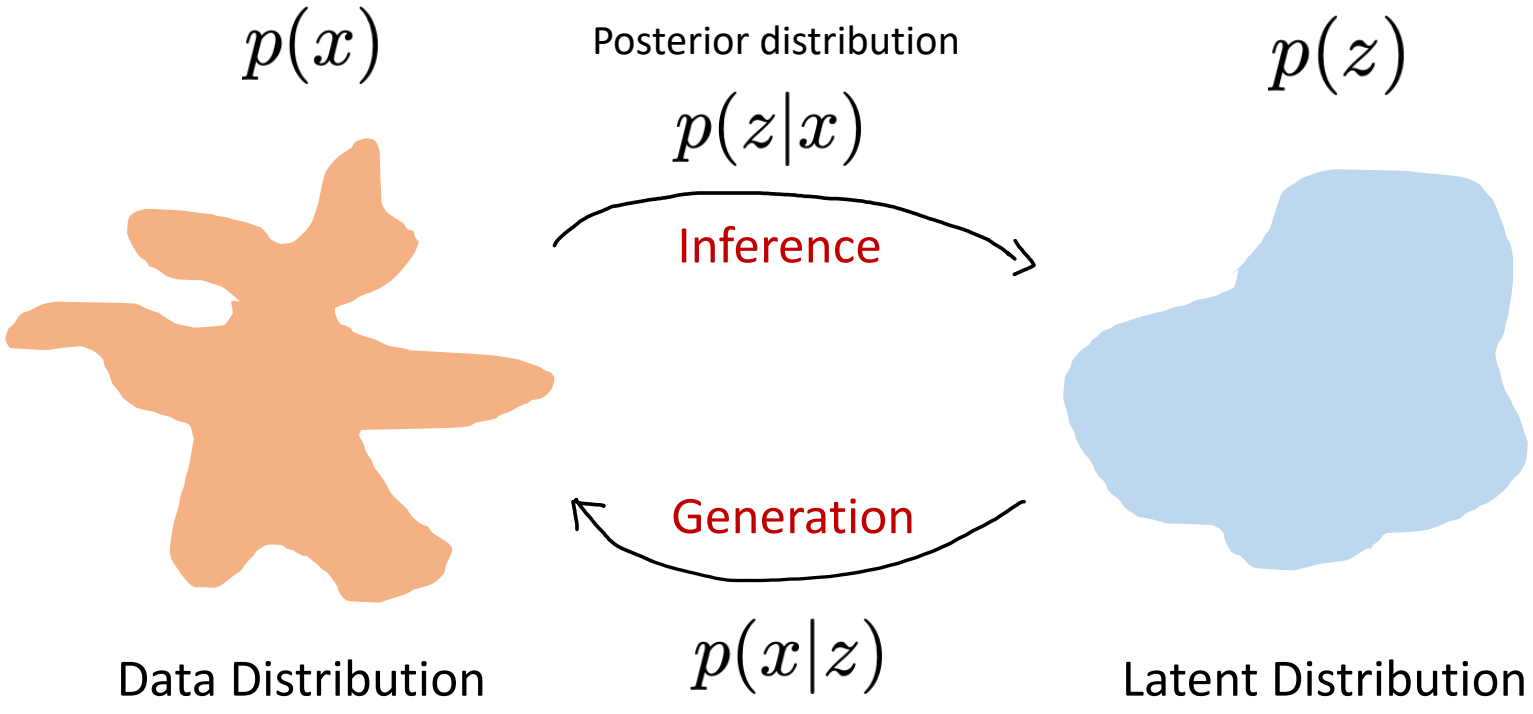






Data Distribution

$$p(x)$$


$p(x)$
Posterior distribution
$p(z|x)$
$p(z)$
Inference
Generation
$p(x|z)$
Data Distribution
Latent Distribution
Likelihood distribution

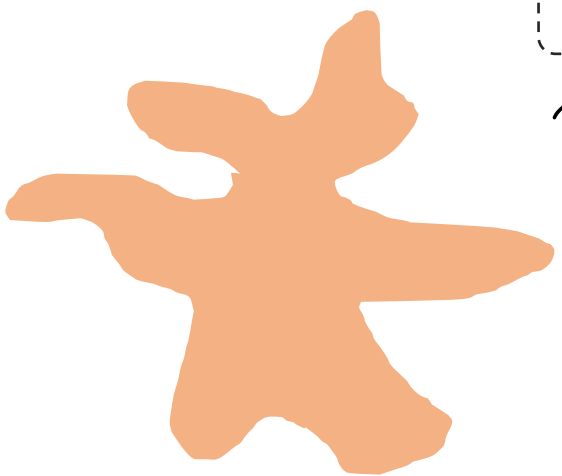
$p(x)$

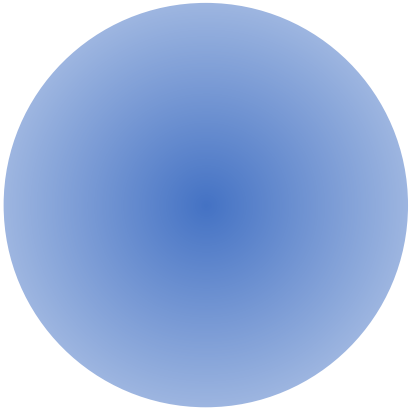
Posterior distribution

$p(z|x)$

gaussian normal distribution

$p(z) = N(0, 1)$





Data Distribution

$p(x|z)$

Likelihood distribution

Latent Distribution

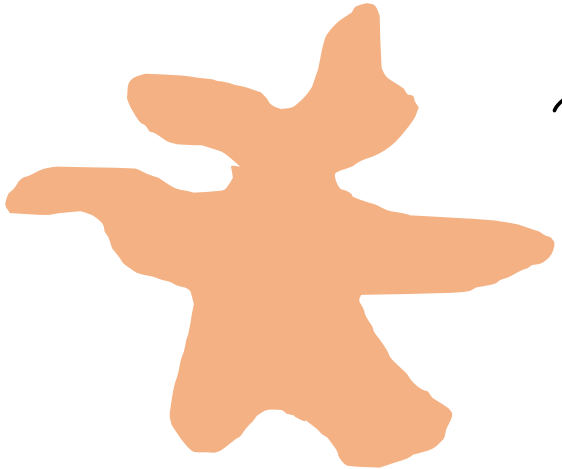
Variational Bayes

gaussian normal distribution

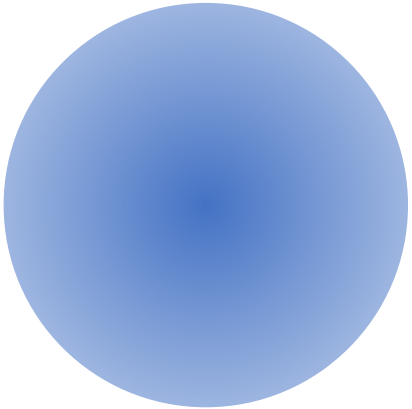
$p(x)$

$q(z|x) \sim p(z|x)$

$p(z) = N(0, 1)$



$p(z|x)$



Data Distribution

$p(x|z)$

Latent Distribution

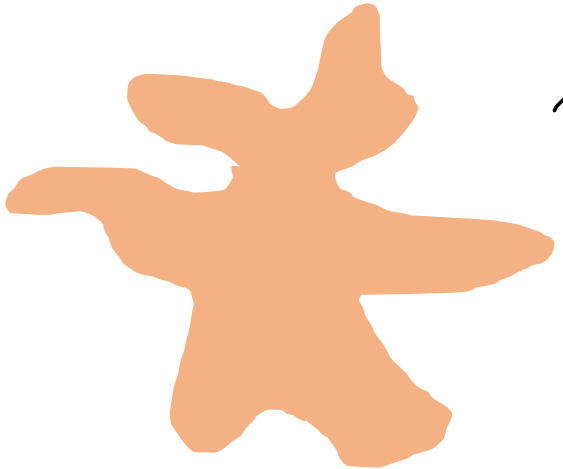
Variational Bayes

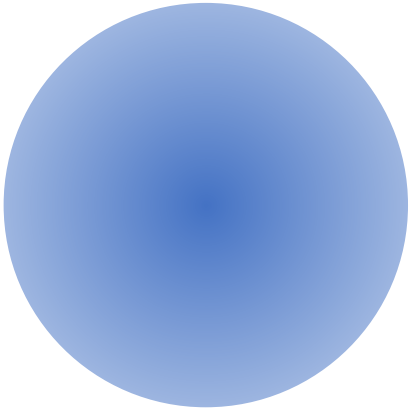
gaussian normal distribution

$p(x)$

$q(z|x) = N(\mu, \sigma^2)$

$p(z) = N(0, 1)$





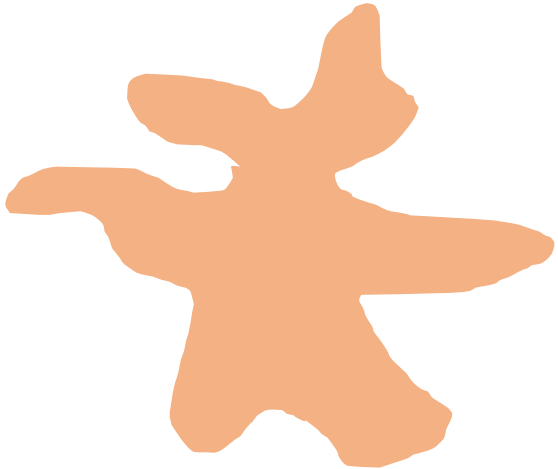
Data Distribution

$p(x|z)$

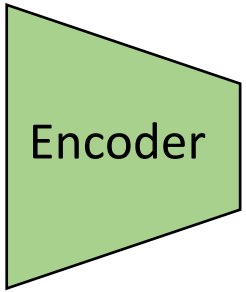
Latent Distribution

Autoencoder Variational Bayes

$p(x)$

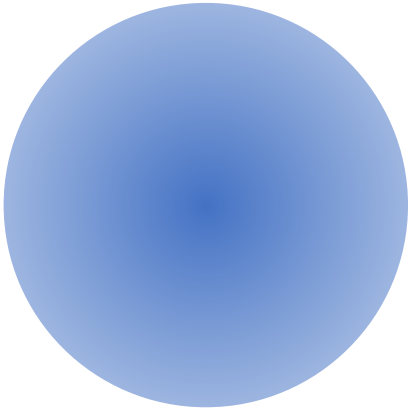


Data Distribution



gaussian normal distribution

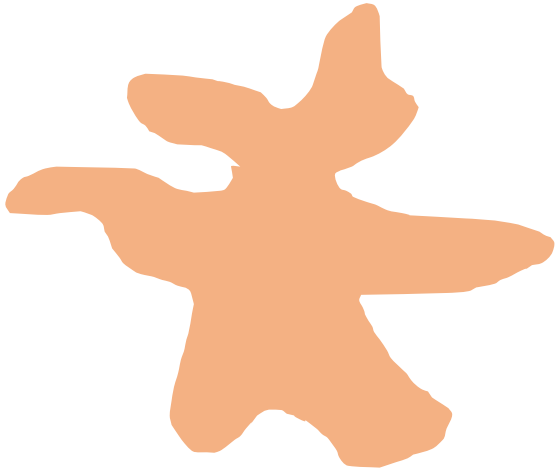
$p(z) = N(0, 1)$



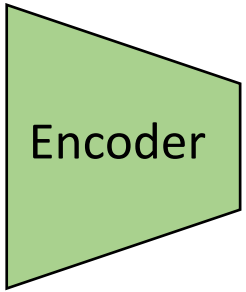
Latent Distribution

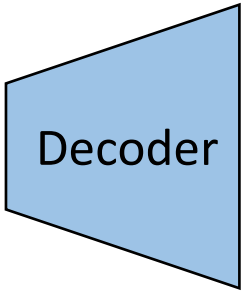
Autoencoder Variational Bayes

$p(x)$



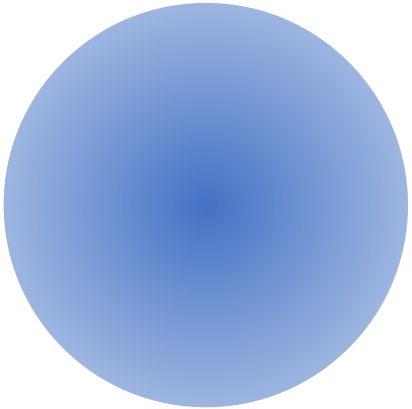
Data Distribution





gaussian normal distribution

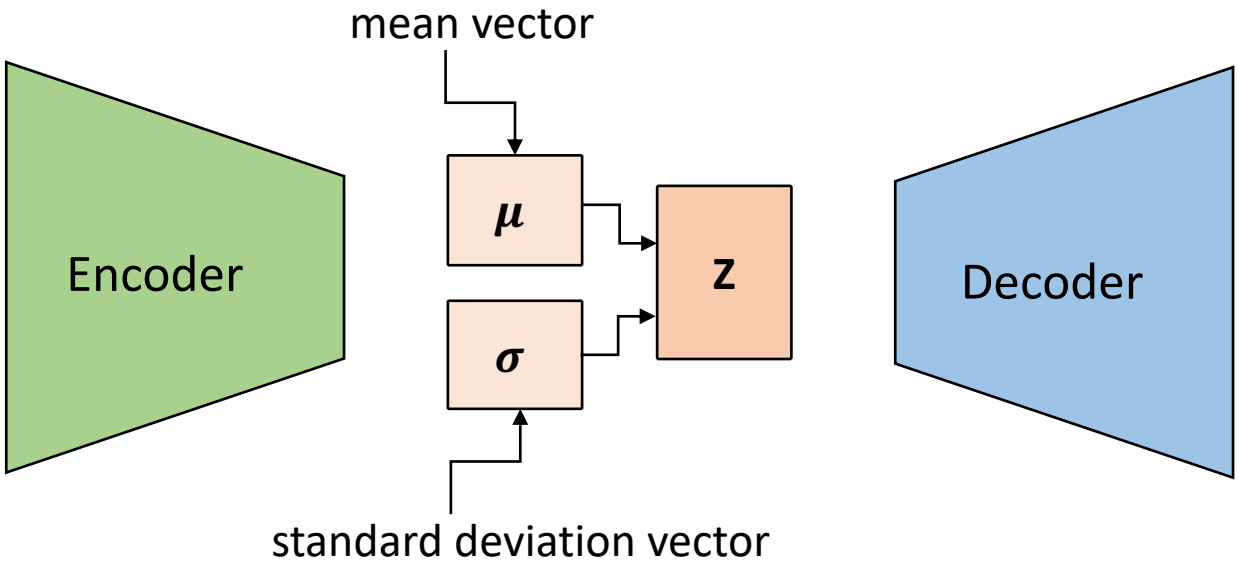
$p(z) = N(0, 1)$



Latent Distribution

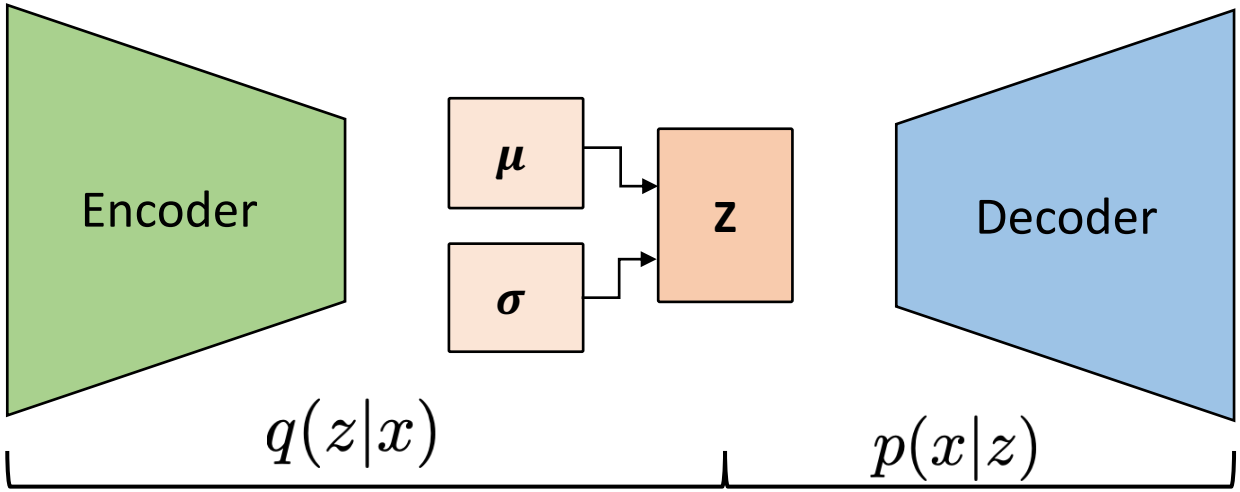
Variational Autoencoders (VAEs)

Sample from the mean and standard deviation to compute latent sample



– by contrast learn a continuous latent space that is easy to sample from
– can generate new data by sampling from the learned generative model

How to train VAEs



Kullback-Leibler (KL) divergence

$$\mathcal{L}(x) = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \mathrm{KL}(q(z|x) \,\|\, p(z))$$

reconstruction loss **regularization term**

VAE is trained to maximize the variational lower bound:

$$\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x}, \mathbf{z}) d\mathbf{z} = \log \int q_\phi(\mathbf{z}|\mathbf{x}) \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} d\mathbf{z}$$

$$= \log \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right]$$

$$\geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \log \left[\frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right]$$

Evidence Lower Bound

Tightness Condition:

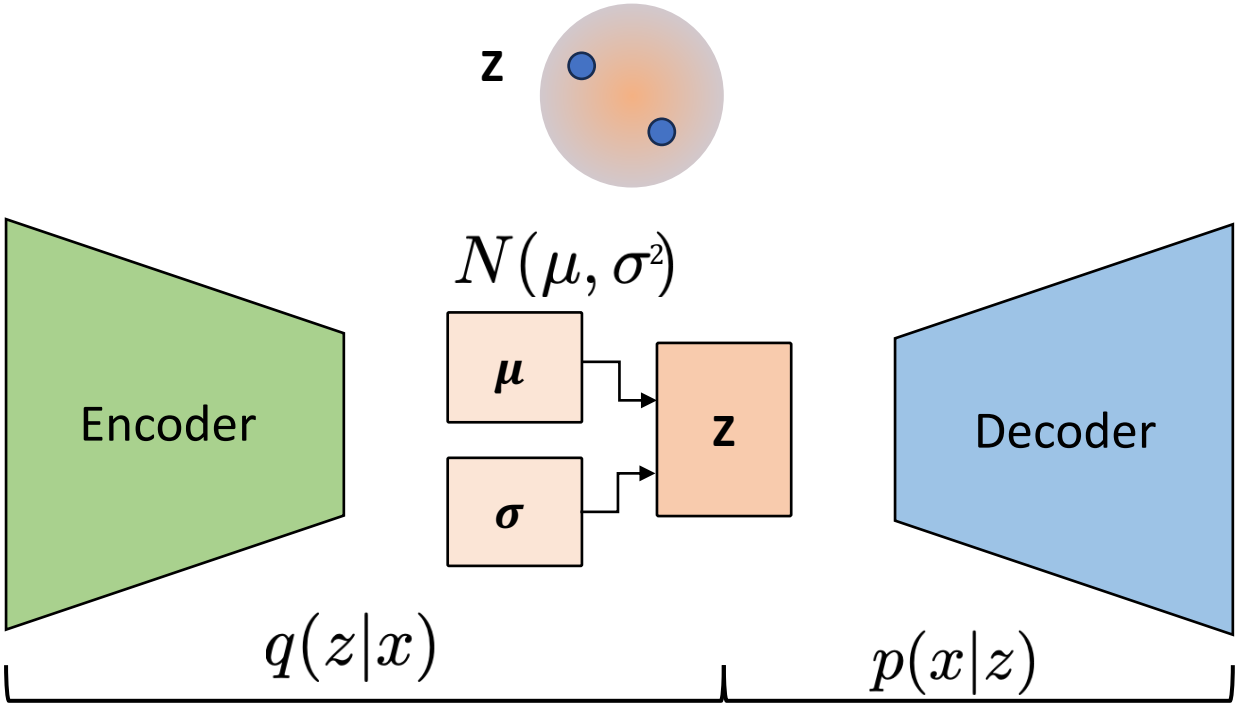
$$q_\phi(\mathbf{z}|\mathbf{x}) = p_\theta(\mathbf{z}|\mathbf{x})$$

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}$$

$$\mathbb{E}[f(x)] \geq f(\mathbb{E}[x])$$

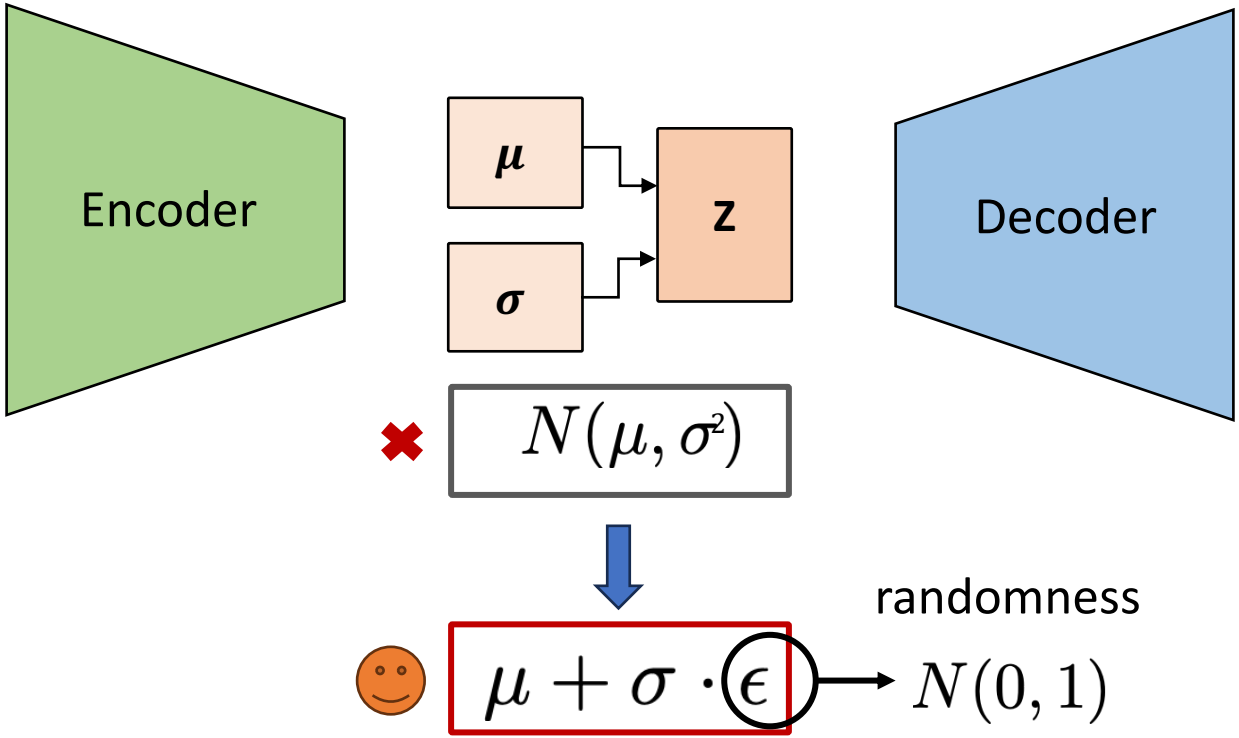
Jensen's inequality

We cannot backpropagate gradients through sampling layers

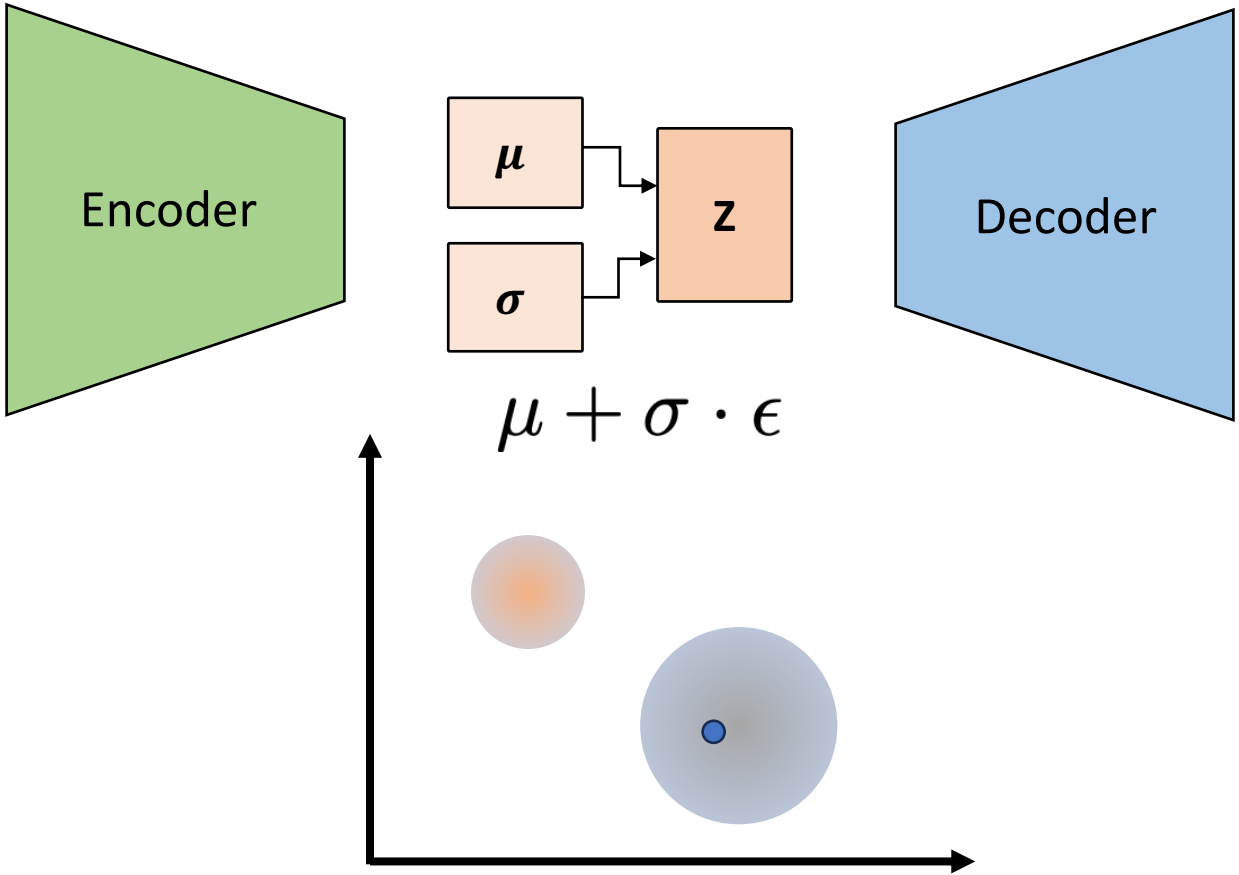


$$\mathcal{L}(x) = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \mathrm{KL}\big(q(z|x) \,\|\, p(z)\big)$$

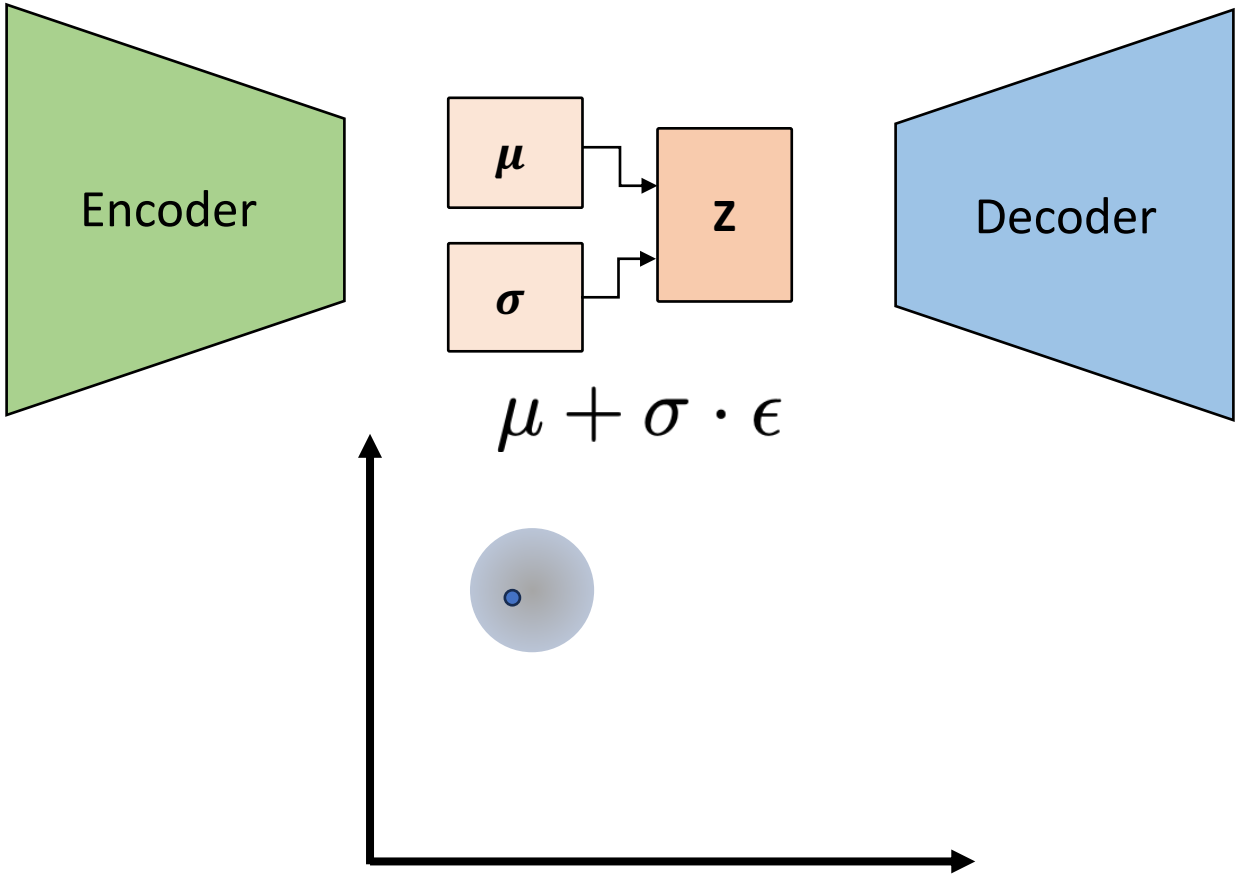
Reparametrizing the sampling layer



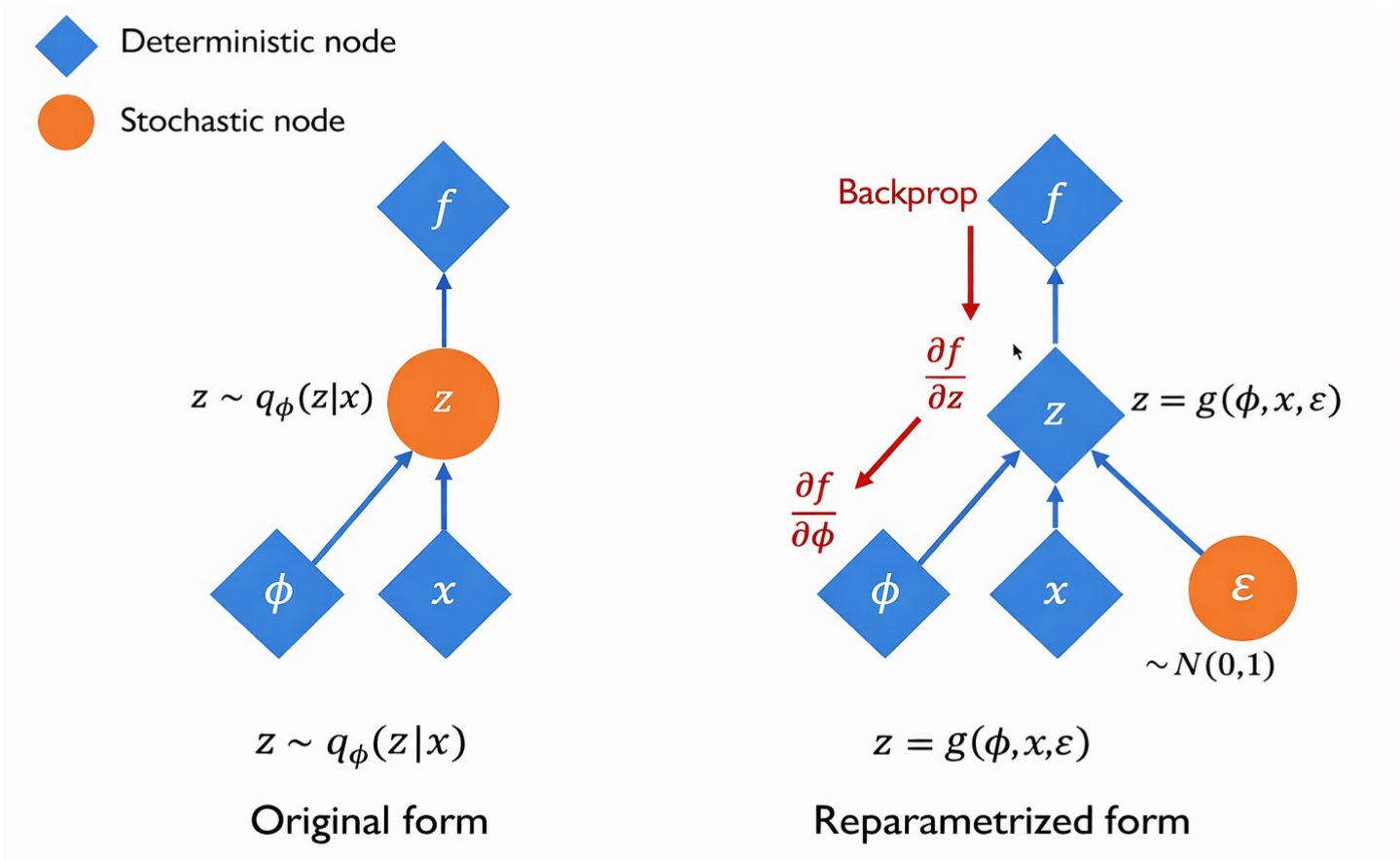
Reparametrizing the sampling layer



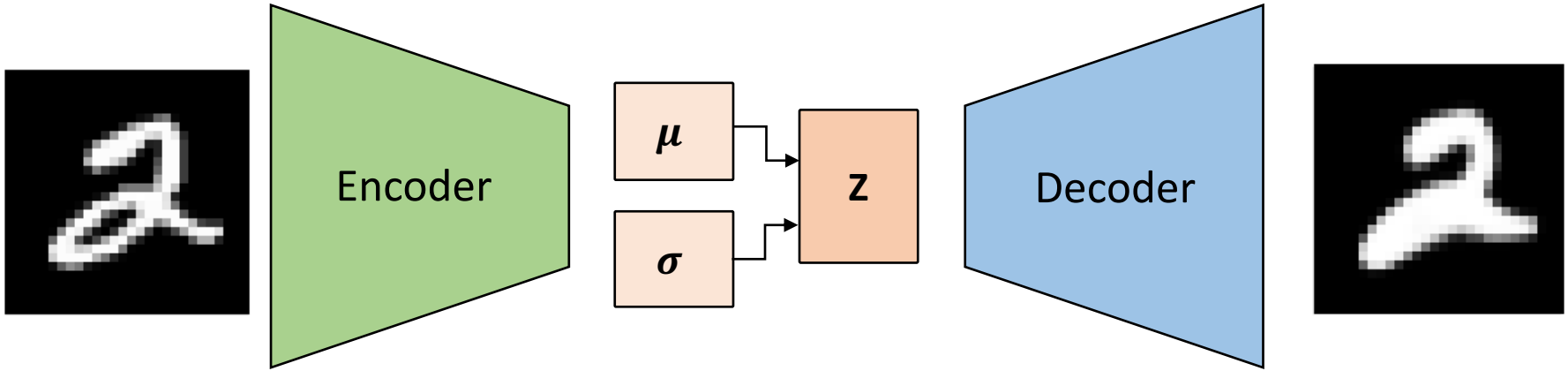
Reparametrizing the sampling layer



Reparametrizing the sampling layer

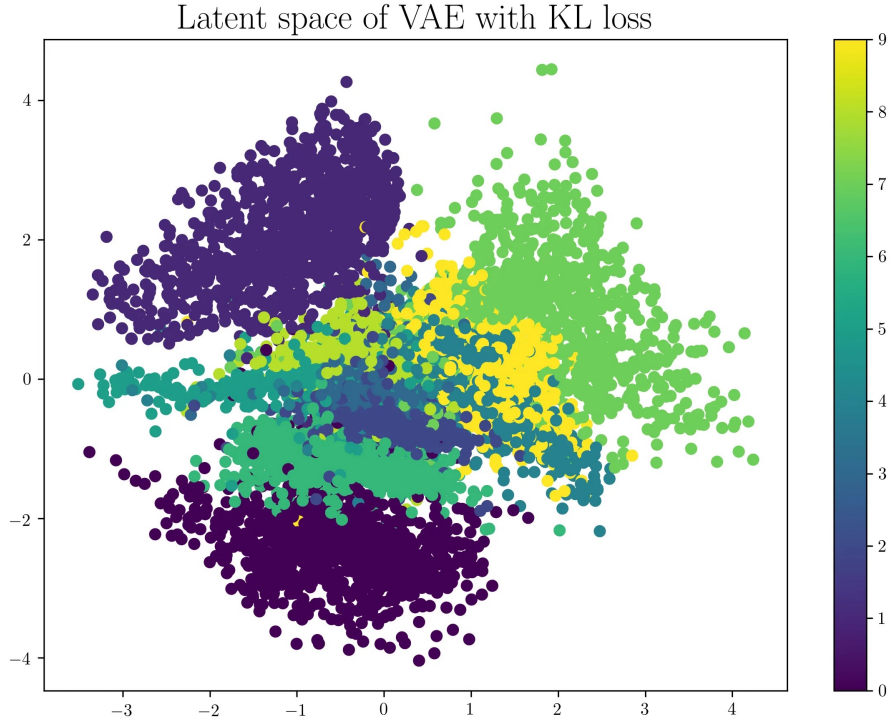


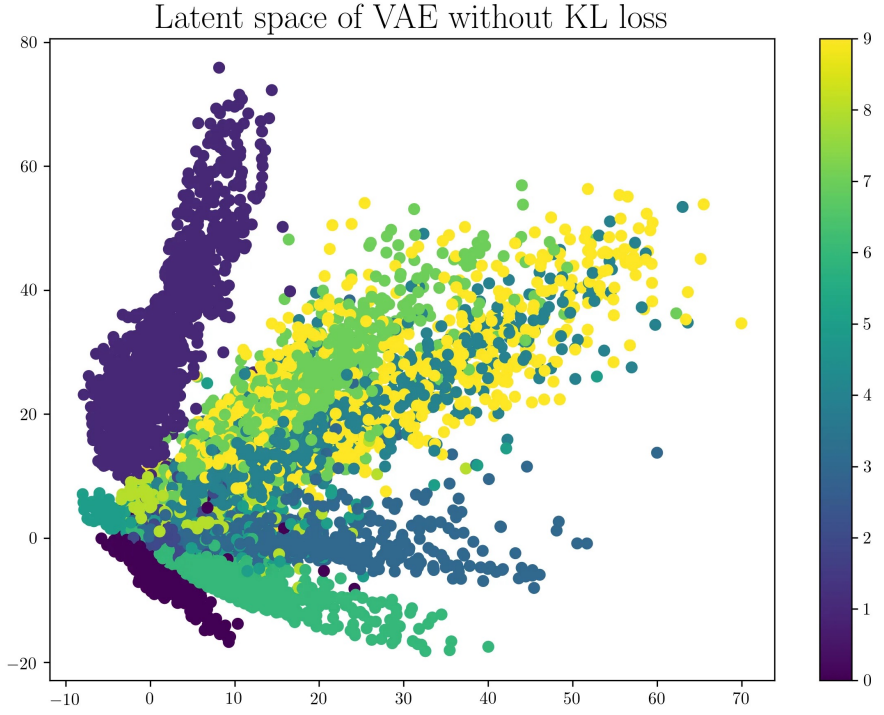
Variational Autoencoders (VAEs)



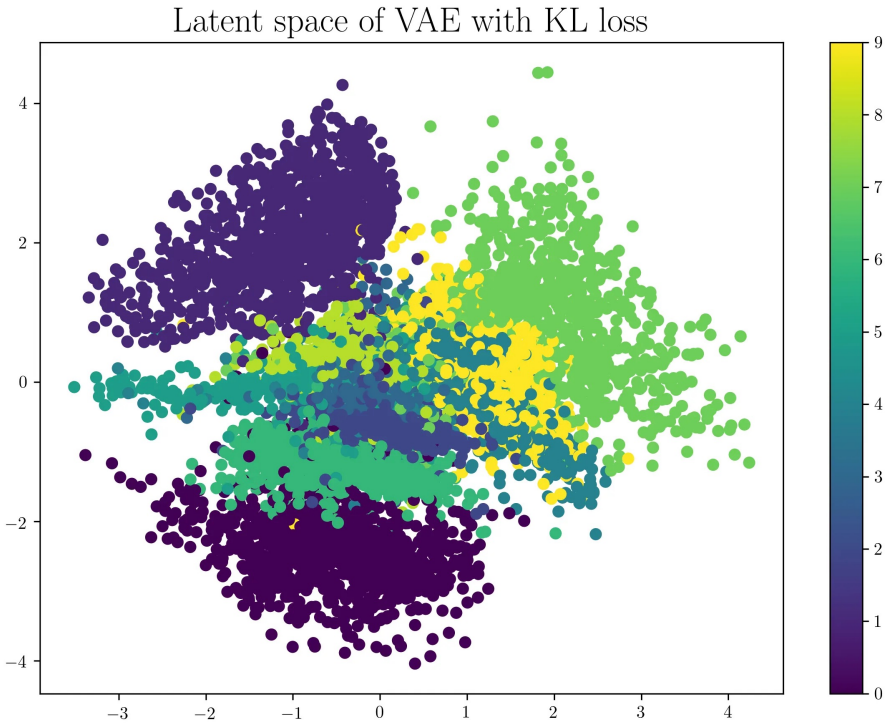
https://xnought.github.io/vae-explainer/

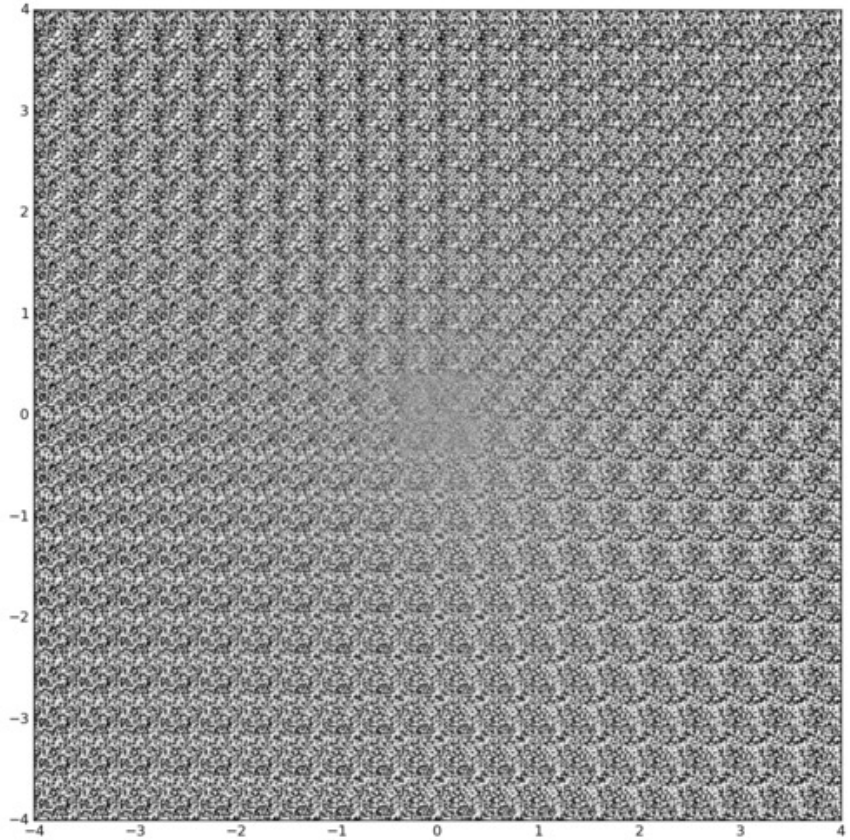
Variational Autoencoders (VAEs)



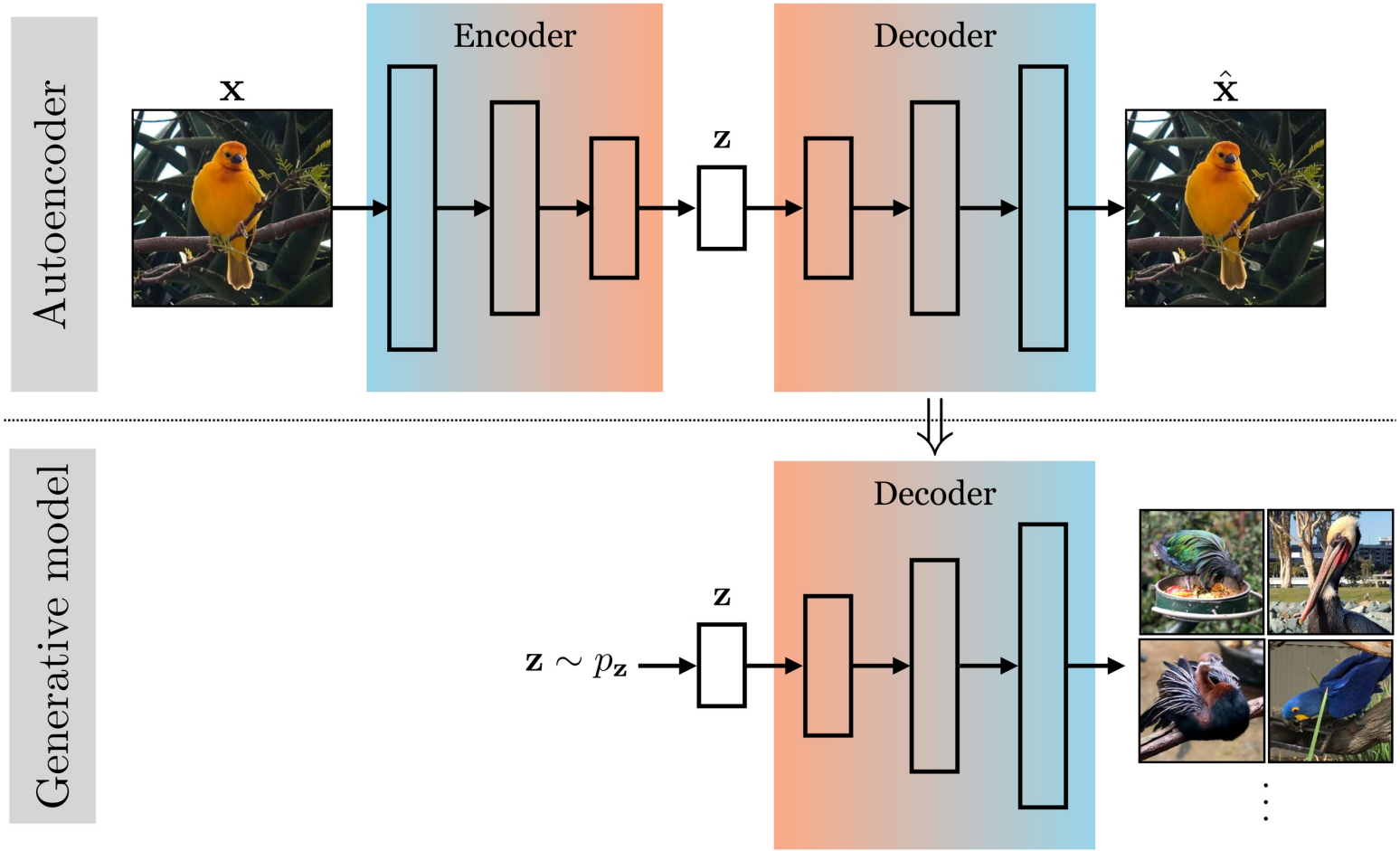


Variational Autoencoders (VAEs)

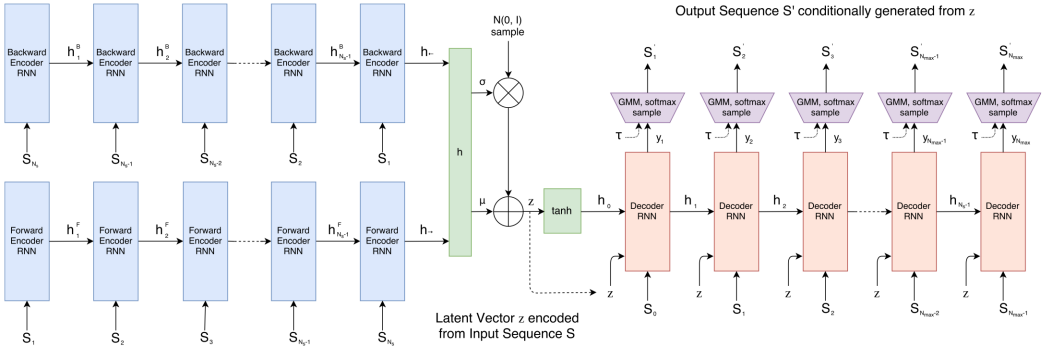




https://tayden.github.io/VAE-Latent-Space-Explorer/

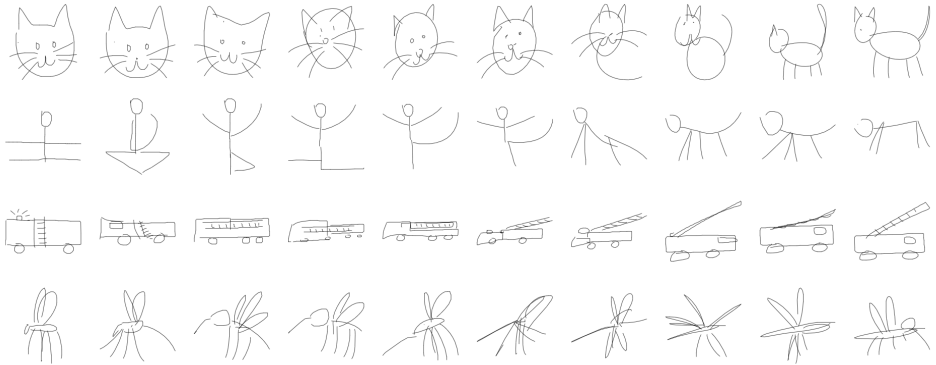

Autoencoder
x
Encoder
z
Decoder
x̂
Generative model
Decoder
z
z ∼ p_z

Sequence-to-Sequence VAEs



A Neural Representation of Sketch Drawings (arXiv:1704.03477)

Latent Space Interpolations



Latent space interpolation of music:

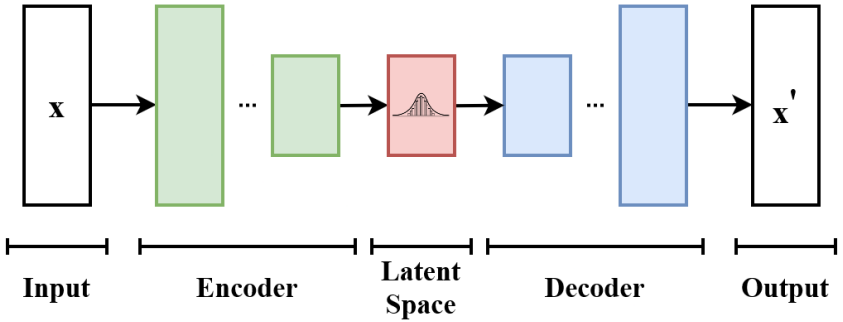
https://magenta.withgoogle.com/music-vae

VAE Examples



VAEs





Learning Structured Output Representation using Deep Conditional Generative Models

Kihyuk Sohn[*†] **Xinchen Yan**[†] **Honglak Lee**[†]
[*] NEC Laboratories America, Inc.
[†] University of Michigan, Ann Arbor
ksohn@nec-labs.com, {xcyan,honglak}@umich.edu

Abstract

Supervised deep learning has been successfully applied to many recognition problems. Although it can approximate a complex many-to-one function well when a large amount of training data is provided, it is still challenging to model complex structured output representations that effectively perform probabilistic inference and make diverse predictions. In this work, we develop a deep conditional generative model for structured output prediction using Gaussian latent variables. The model is trained efficiently in the framework of stochastic gradient variational Bayes, and allows for fast prediction using stochastic feed-forward inference. In addition, we provide novel strategies to build robust structured prediction algorithms, such as input noise-injection and multi-scale prediction objective at training. In experiments, we demonstrate the effectiveness of our proposed algorithm in comparison to the deterministic deep neural network counterparts in generating diverse but realistic structured output predictions using stochastic inference. Furthermore, the proposed training methods are complimentary, which leads to strong pixel-level object segmentation and semantic labeling performance on Caltech-UCSD Birds 200 and the subset of Labeled Faces in the Wild dataset.

$$\mathcal{L} = \frac{1}{N}\sum_{n=1}^{N} \mathbb{E}_q[\log p(x^{(n)}|z)] - \beta \mathrm{KL}[q(z|x^{(n)})||p(z)],$$

Published as a conference paper at ICLR 2017

β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework

Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, Alexander Lerchner
Google DeepMind
{irinah,lmatthey,arkap,cpburgess,glorotx, botvinick,shakir,lerchner}@google.com

Abstract

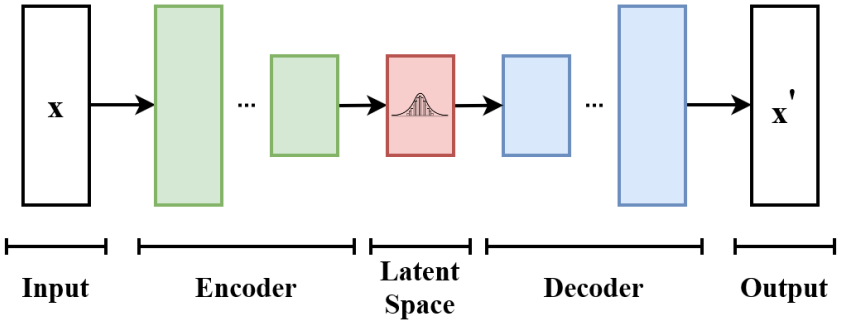
Learning an interpretable factorised representation of the independent data generative factors of the world without supervision is an important precursor for the development of artificial intelligence that is able to learn and reason in the same way that humans do. We introduce β-VAE, a new state-of-the-art framework for automated discovery of interpretable factorised latent representations from raw image data in a completely unsupervised manner. Our approach is a modification of the variational autoencoder (VAE) framework. We introduce an adjustable hyperparameter β that balances latent channel capacity and independence constraints with reconstruction accuracy. We demonstrate that β-VAE with appropriately tuned $\beta > 1$ qualitatively outperforms VAE ($\beta = 1$), as well as state of the art unsupervised (InfoGAN) and semi-supervised (DC-IGN) approaches to disentangled factor learning on a variety of datasets (*celebA*, *faces* and *chairs*). Furthermore, we devise a protocol to quantitatively compare the degree of disentanglement learnt by different models, and show that our approach also significantly outperforms all baselines quantitatively. Unlike InfoGAN, β-VAE is stable to train, makes few assumptions about the data and relies on tuning a single hyperparameter β, which can be directly optimised through a hyperparameter search using weakly labelled data or through heuristic visual inspection for purely unsupervised data.

VAE Examples



VAEs





VQ-VAEs

Neural Discrete Representation Learning

Aaron van den Oord
DeepMind
avdnoord@google.com

Oriol Vinyals
DeepMind
vinyals@google.com

Koray Kavukcuoglu
DeepMind
korayk@google.com

Abstract

Learning useful representations without supervision remains a key challenge in machine learning. In this paper, we propose a simple yet powerful generative model that learns such discrete representations. Our model, the Vector Quantised-Variational AutoEncoder (VQ-VAE), differs from VAEs in two key ways: the encoder network outputs discrete, rather than continuous, codes; and the prior is learnt rather than static. In order to learn a discrete latent representation, we incorporate ideas from vector quantisation (VQ). Using the VQ method allows the model to circumvent issues of "posterior collapse" -— where the latents are ignored when they are paired with a powerful autoregressive decoder -— typically observed in the VAE framework. Pairing these representations with an autoregressive prior, the model can generate high quality images, videos, and speech as well as doing high quality speaker conversion and unsupervised learning of phonemes, providing further evidence of the utility of the learnt representations.

VAE Examples

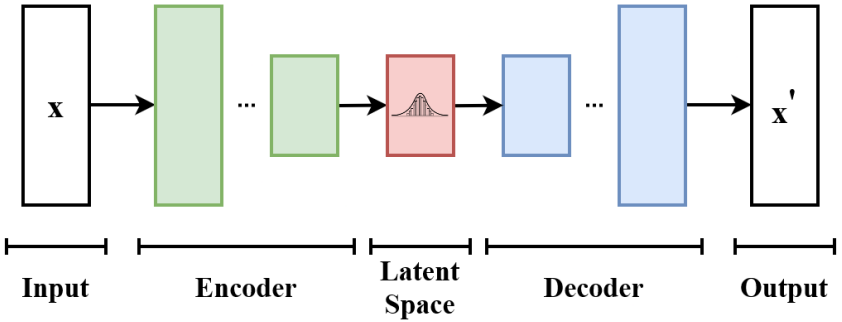


VAEs



GANs





Generative Adversarial Networks (GANs)

arXiv.org > stat > arXiv:1406.2661

Search...

Help | Advanced

Statistics > Machine Learning

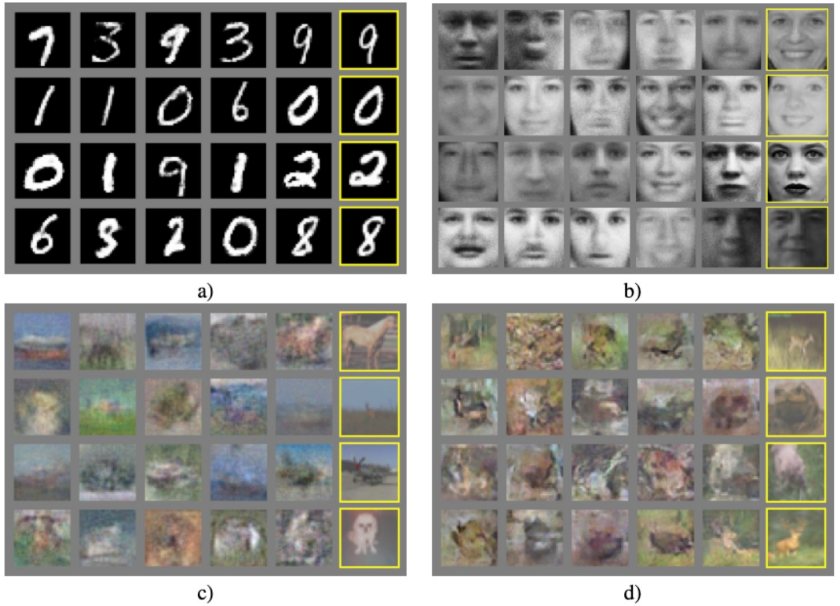
[Submitted on 10 Jun 2014]

Generative Adversarial Networks

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, Yoshua Bengio

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G. The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D, a unique solution exists, with G recovering the training data distribution and D equal to 1/2 everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

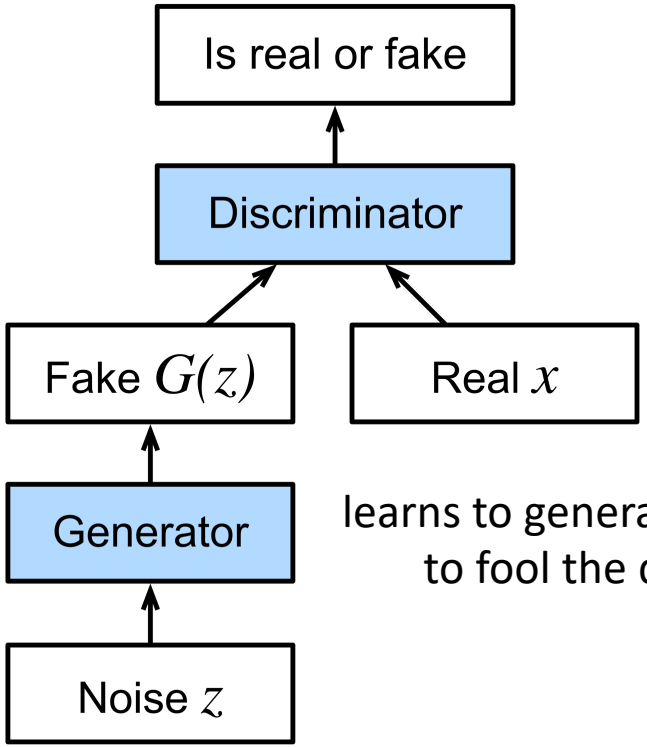
https://arxiv.org/abs/1406.2661



https://thispersondoesnotexist.com

Generative Adversarial Networks (GANs)

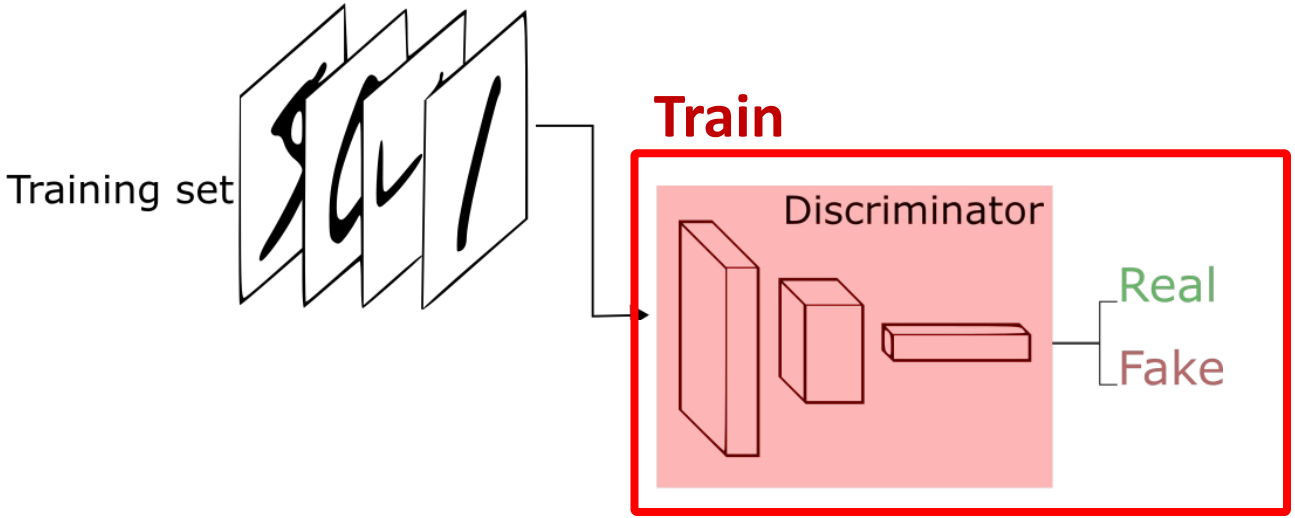
Adversarial Game



learns to become better at distinguishing real from generated images/data

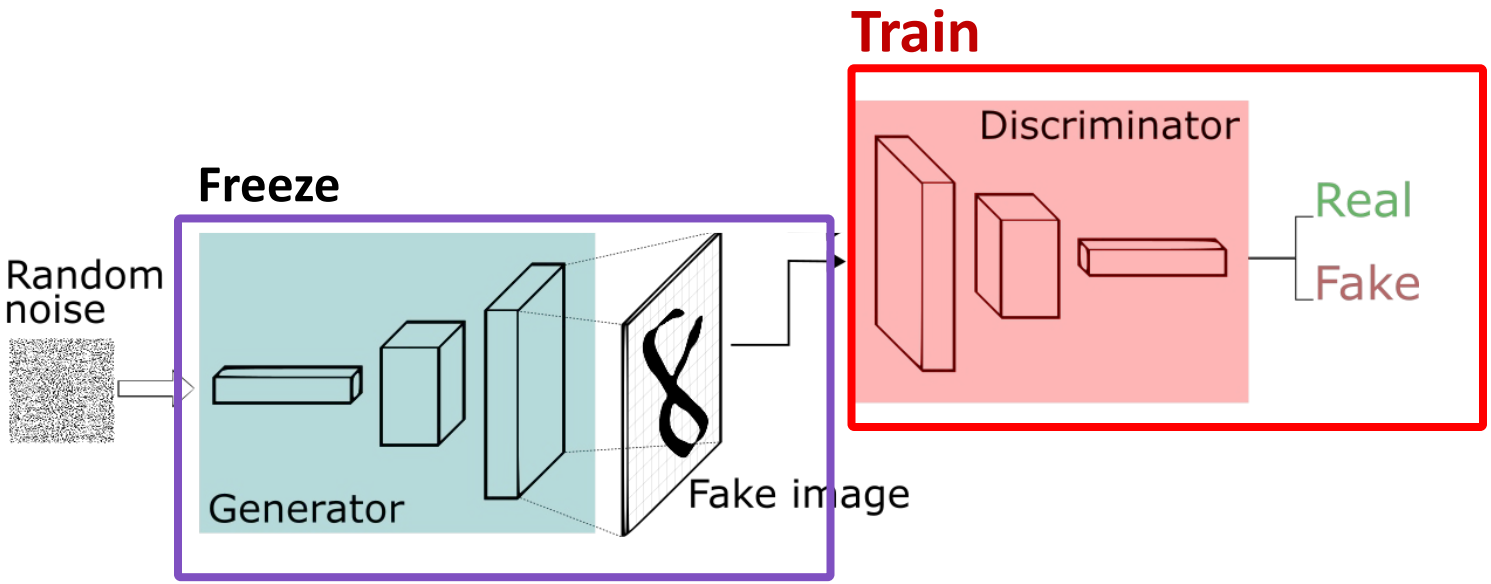
learns to generate better images to fool the discriminator

Train Discriminator



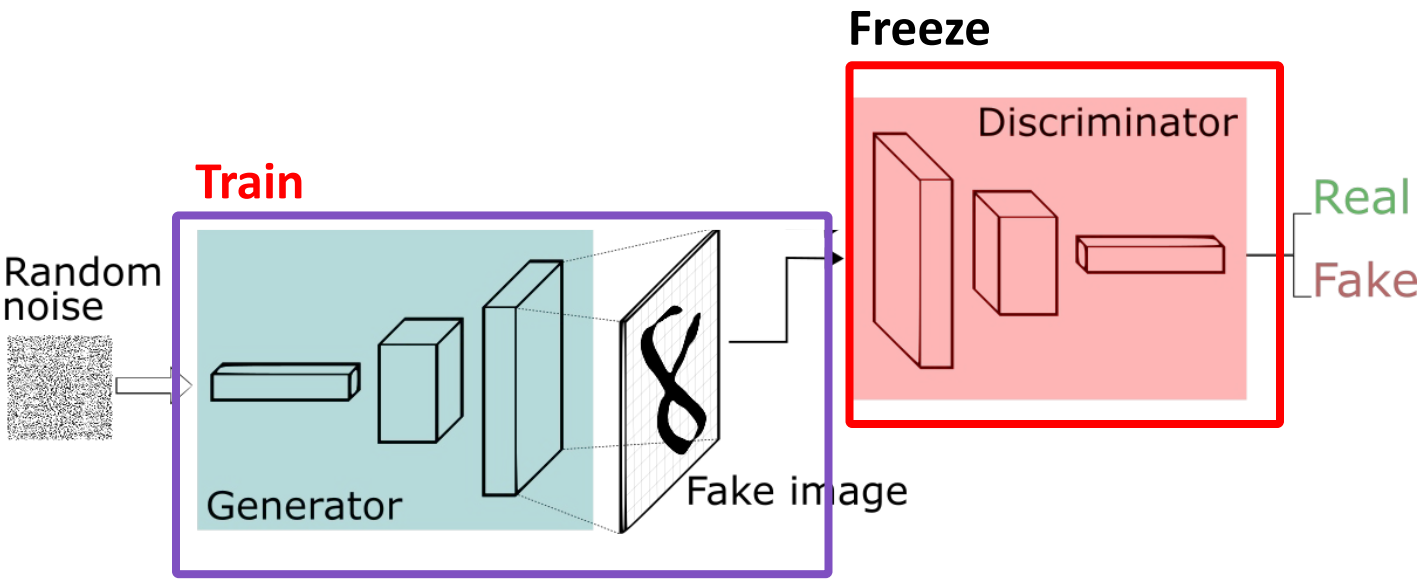
Train to predict that real image is real

Train Discriminator



Train to predict that fake image is fake

Train Generator

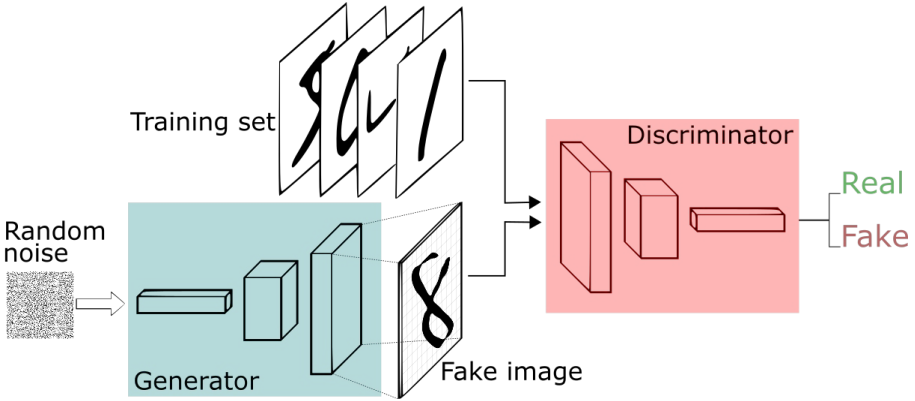


Train to predict that fake image is real

How do the loss functions look like?

Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}\ (\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$



Imagine fixing G

Discriminator wants D(x) = 1 for real data

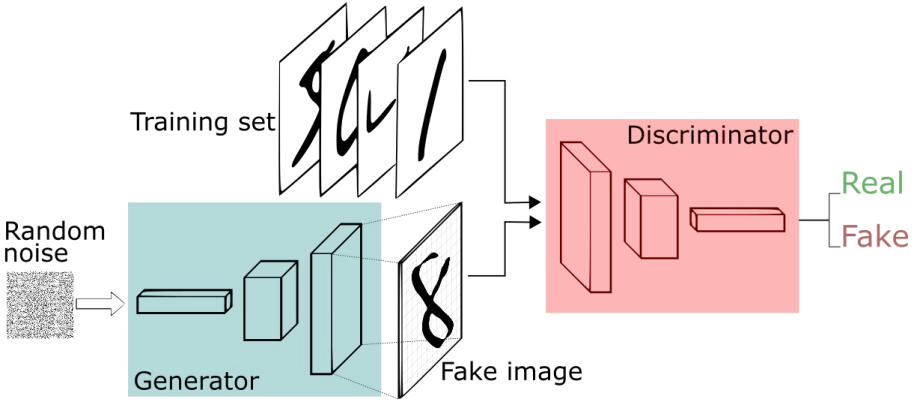
Discriminator wants D(x) = 0 for fake data

$$\min_G \max_D \left(E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p(z)}\left[\log\left(1 - D\left(G(z)\right)\right)\right]\right)$$

How do the loss functions look like?

Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}\ (\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$



This term does not depend on G

Generator wants D(x) = 1 for fake data

Imagine fixing **D**

$$\min_{\boldsymbol{G}} \max_{D} \left(E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p(z)}\left[\log\left(1 - D\left(G(z)\right)\right)\right] \right)$$

How do the loss functions look like?

Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}\ (\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Imagine fixing G

Discriminator wants D(x) = 1 for real data

Discriminator wants D(x) = 0 for fake data

$$\min_G \max_D \left(E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p(z)}\left[\log\left(1 - D\left(G(z)\right)\right)\right]\right)$$

Imagine fixing D

This term does not depend on G

Generator wants D(x) = 1 for fake data

$$\min_G \max_D \left(E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p(z)}\left[\log\left(1 - D\left(G(z)\right)\right)\right]\right)$$

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)\right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

  end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Simplified GAN Loss

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}\ (\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Minimize Discriminator Loss

Real image

Generated image

$$\min_D \ -(E_{x\sim p_x}[\log D(x)] + E_{z\sim p_z}[\log(1 - D(G(z)))])$$

using "original" labels

Real images: class label 1
Generated images: class label 0

Sebastian Raschka, summer school on deep learning 2021

Simplified GAN Loss

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}\ (\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Minimize Discriminator Loss

Real image

Generated image

$$\min_D \ -(E_{x\sim p_x}[\log D(x)] + E_{z\sim p_z}[\log(1 - D(G(z)))])$$

value range: $[0, 1]$

want it to predict *real* -> 1
(correct prediction)

$$\log D(x)$$

value range: $[-\infty, 0]$

$$-(E_{x\sim p_x}[\log D(x)])$$

loss value range: $[\infty, 0]$

Simplified GAN Loss

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}\ (\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Minimize Discriminator Loss

Real image

Generated image

$$\min_D \ -(E_{x\sim p_x}[\log D(x)] + E_{z\sim p_z}[\log(1 - D(G(z)))])$$

value range: $[0, 1]$

predict *generated* -> 0
(correct prediction)

$$1 - D(G(z))$$

value range: $[1, 0]$

$$-(E_{z\sim p_z}[\log(1 - D(G(z)))])$$

loss value range: $[0, \infty]$

Simplified GAN Loss

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}\ (\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Minimize Generator Loss

Generated image

$$\min_G \ -(E_{z\sim p_z}[\log D(G(z))])$$

Want to fool discriminator to make a wrong prediction

Generated images: predict class label 1

Simplified GAN Loss

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}\ (\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Minimize Generator Loss

Generated image

$$\min_G \ -(E_{z\sim p_z}[\log D(G(z))])$$

value range: $[0, 1]$

want it to ~~predict *generated* -> 0~~
predict real -> 1

$$\log D(G(z))$$

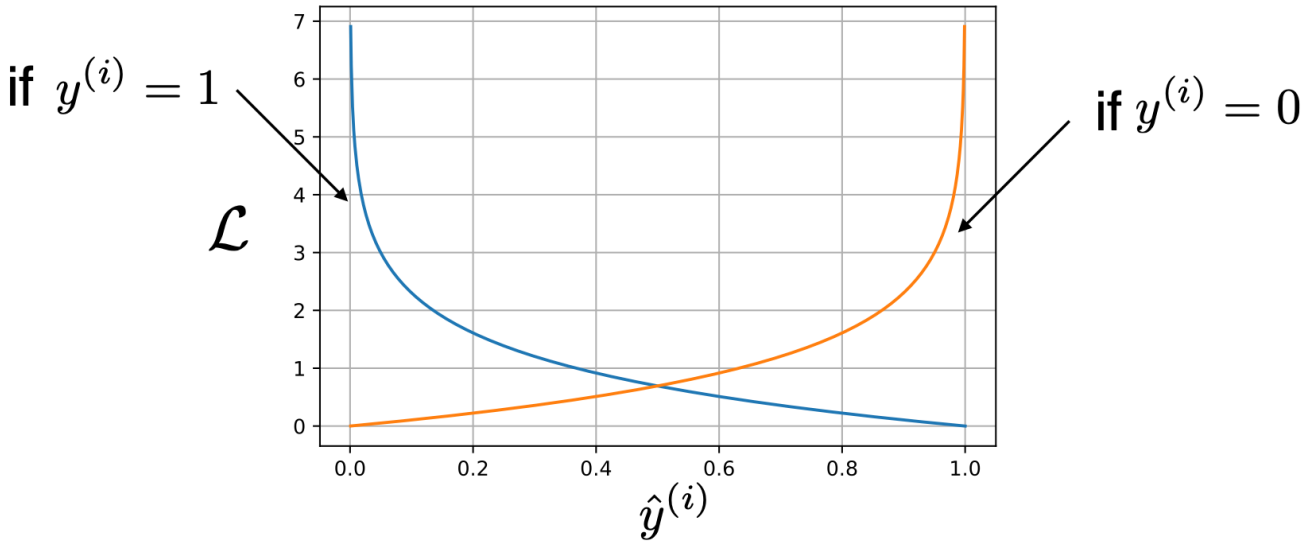
value range: $[-\infty, 0]$

$$-(E_{x\sim p_x}[\log D(G(z))])$$

loss value range: $[\infty, 0]$

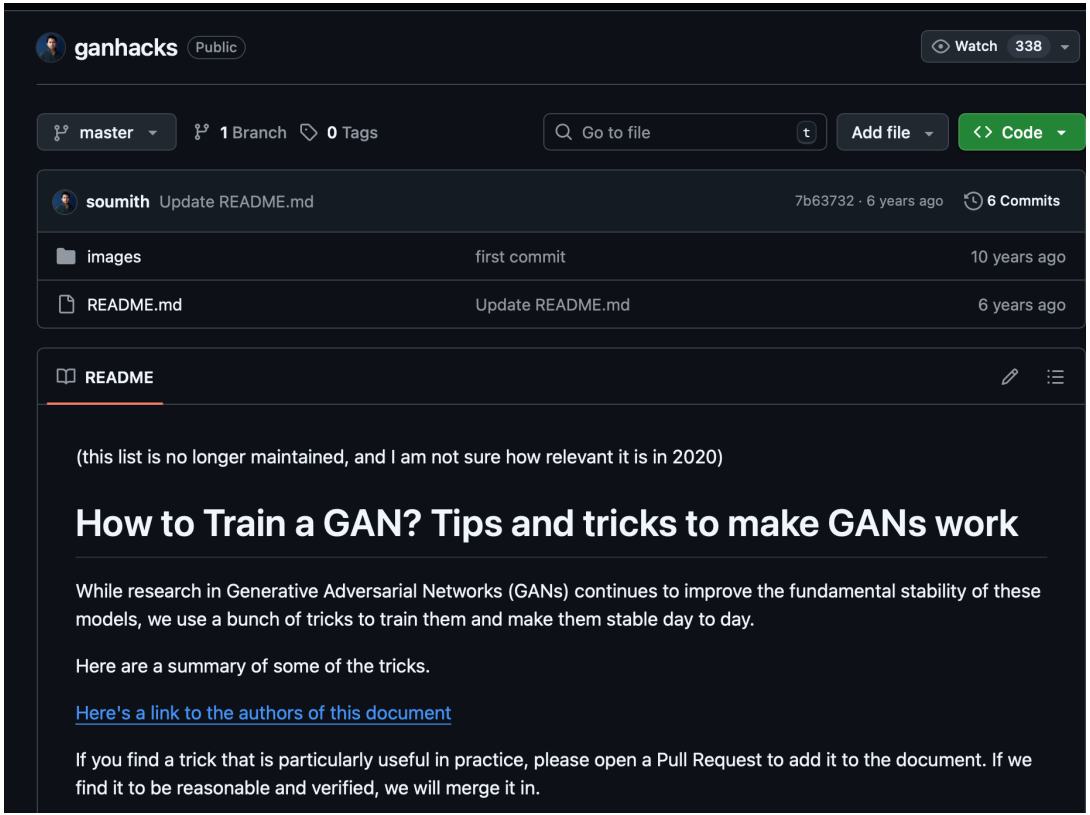
Simplified GAN Loss

Negative Log-Likelihood / Binary Cross Entropy Loss



$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}\left[-\left(y^{(i)}\log\left(\hat{y}^{(i)}\right) + \left(1-y^{(i)}\right)\log\left(1-\hat{y}^{(i)}\right)\right)\right]$$

https://github.com/soumith/ganhacks

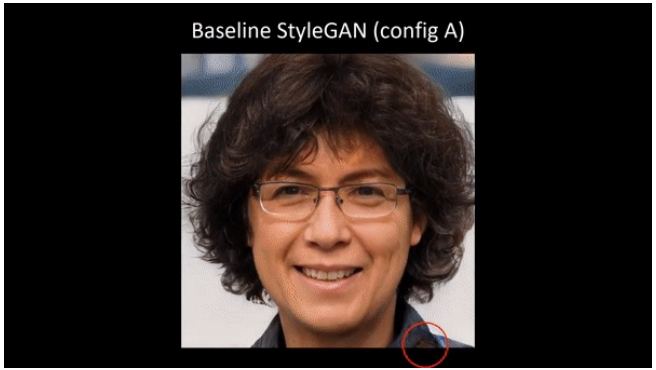


Cycle-GANs



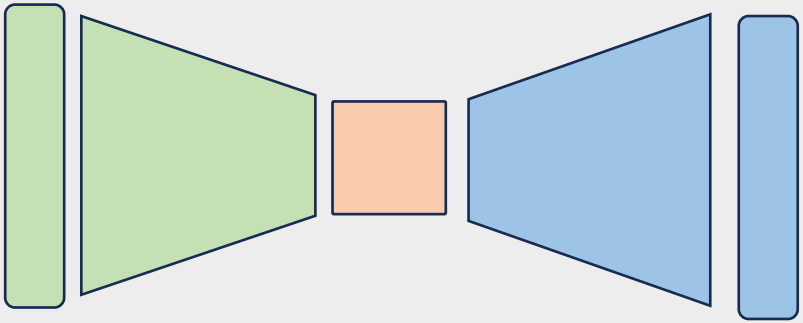
Style-GANs





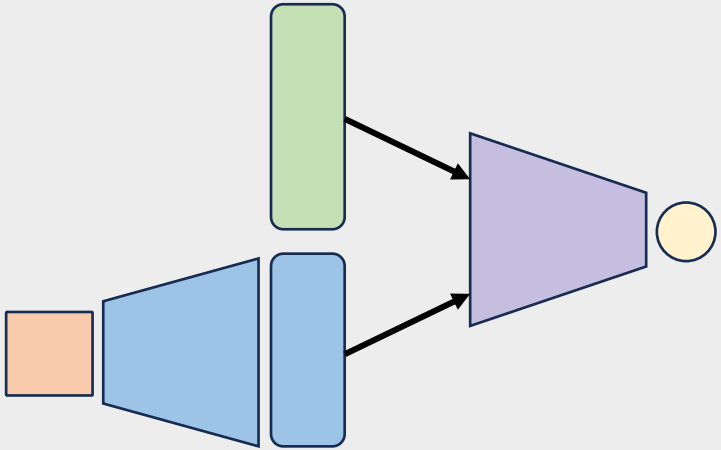
Autoencoder and **Variational Autoencoders (VAEs)**

Learn lower-dimensional **latent space** and **sample** to generate input reconstructions



Generative Adversarial Networks (GANs)

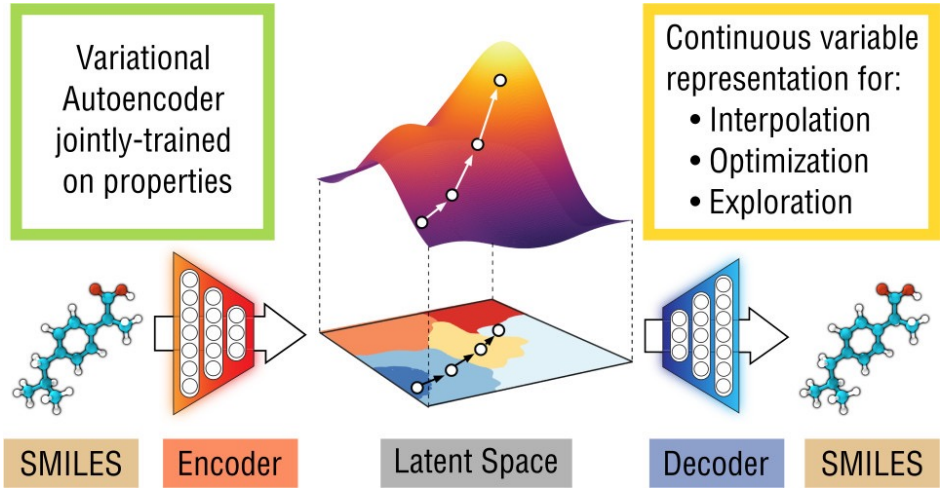
Competing **generator** and **discriminator** networks

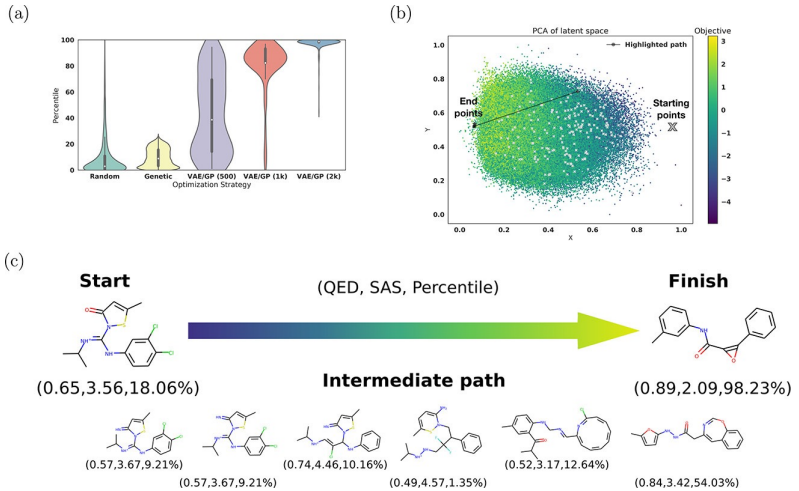


Applications in Science

Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules

https://pubs.acs.org/doi/10.1021/acscentsci.7b00572



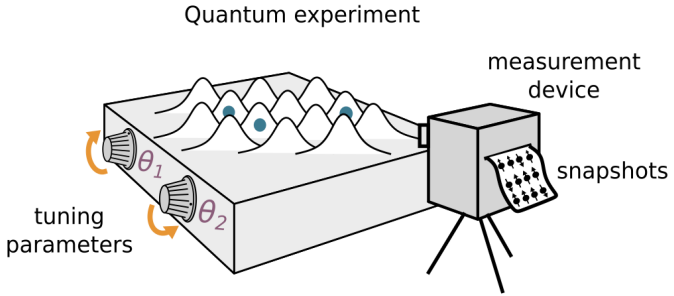


Interpretable representation learning of quantum data

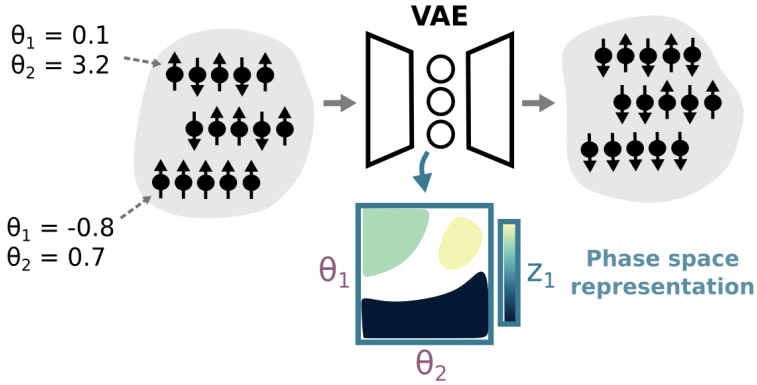
Phys. Rev. A **112**, 062423 (2025) https://doi.org/10.1103/cwb8-y25k

$$\mathcal{L} = \frac{1}{N}\sum_{n=1}^{N} \mathbb{E}_q[\log p(x^{(n)}|z)] - \beta \mathrm{KL}[q(z|x^{(n)})||p(z)],$$

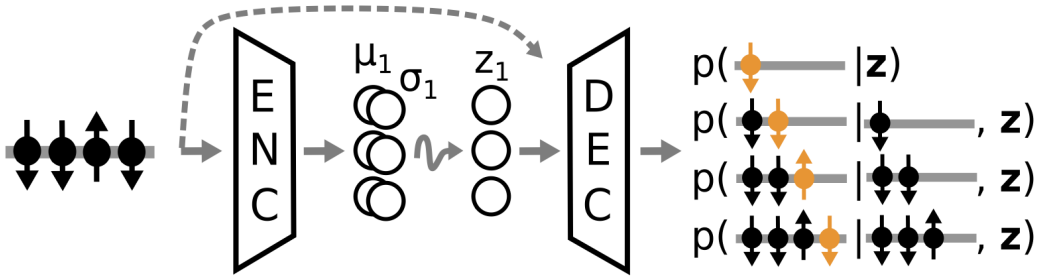
(a) Experimental pipeline



(b) VAE pipeline

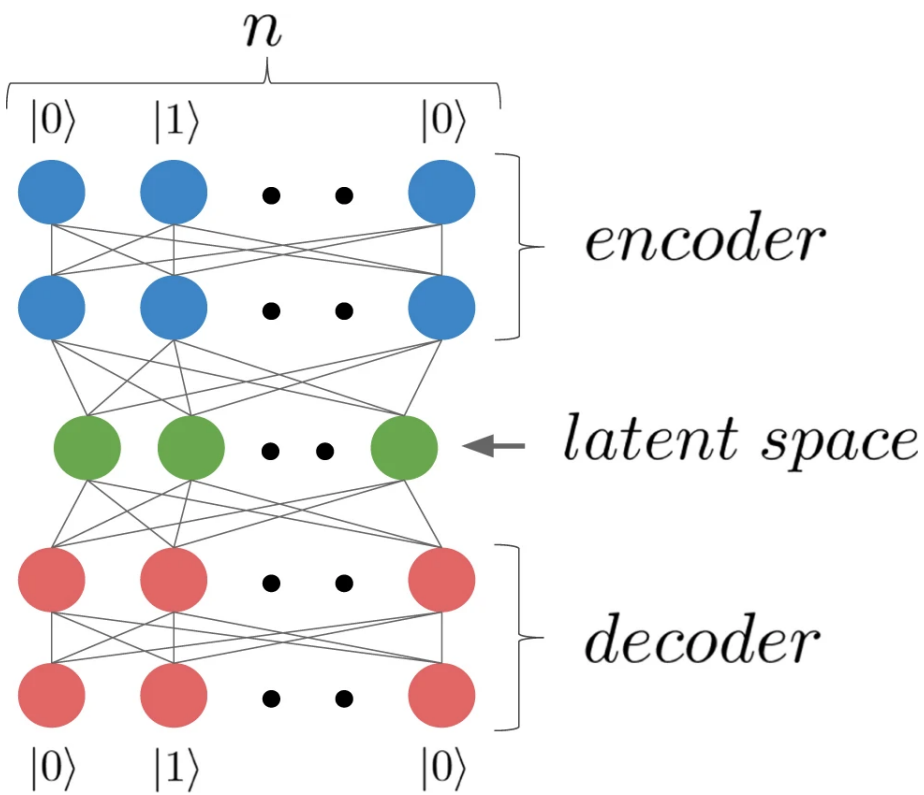


(c) VAE with probabilistic decoder



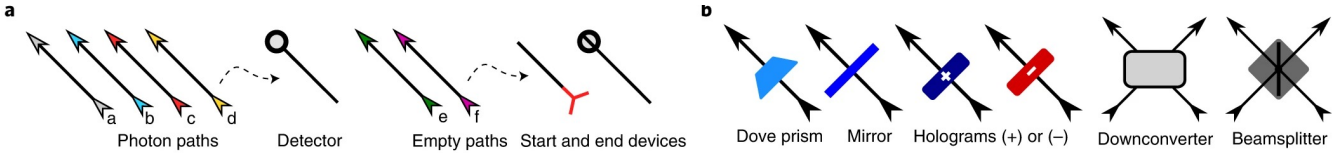
Learning quantum distributions with VAEs

https://doi.org/10.1038/s41534-018-0077-z



Learning interpretable representations of entanglement

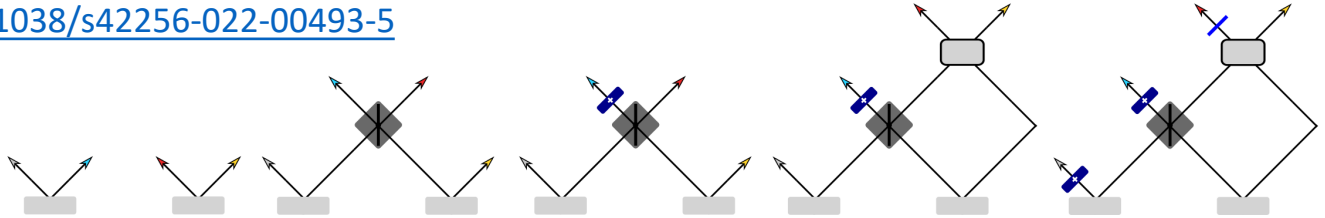
https://doi.org/10.1038/s42256-022-00493-5

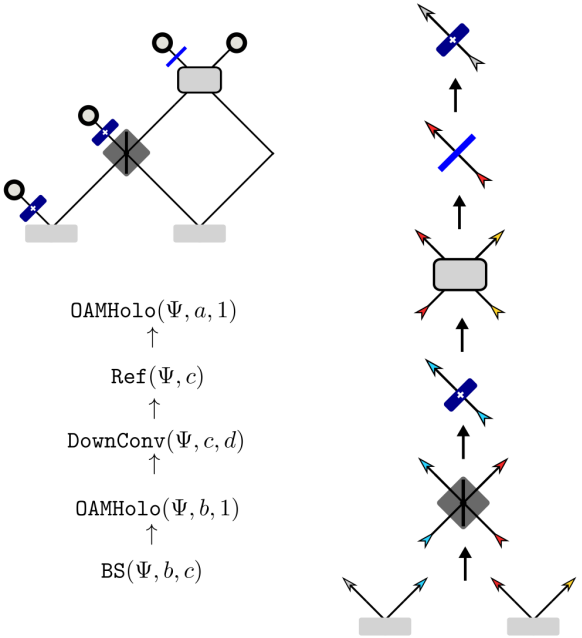


Device	Token	Visual	Operation	Operator
Down Conversion	$\texttt{DownConv}(\Psi, p, p')$		$\|\Psi\rangle + \sum_\ell \|\ell\rangle_p \|-\ell\rangle_{p'}$	$\mathrm{DC}_{pp'}$
Beam Splitter	$\texttt{BS}(\Psi, p, p')$		$\|\ell\rangle_p \longrightarrow \frac{\|\ell\rangle_{p'} + i\|-\ell\rangle_p}{\sqrt{2}}$	$\mathrm{BS}_{pp'}$
Mirror	$\texttt{Ref}(\Psi, p)$		$\|\ell\rangle_p \longrightarrow i\|-\ell\rangle_p$	R_p
Dove Prism	$\texttt{DP}(\Psi, p)$		$\|\ell\rangle_p \longrightarrow ie^{i\pi\ell}\|-\ell\rangle_p$	DP_p
Hologram	$\texttt{OAMHolo}(\Psi, p, n)$		$\|\ell\rangle_p \longrightarrow \|\ell + n\rangle_p$	H_p^n
Hologram	$\texttt{OAMHolo}(\Psi, p, -n)$		$\|\ell\rangle_p \longrightarrow \|\ell - n\rangle_p$	H_p^{-n}

Learning interpretable representations of entanglement

https://doi.org/10.1038/s42256-022-00493-5





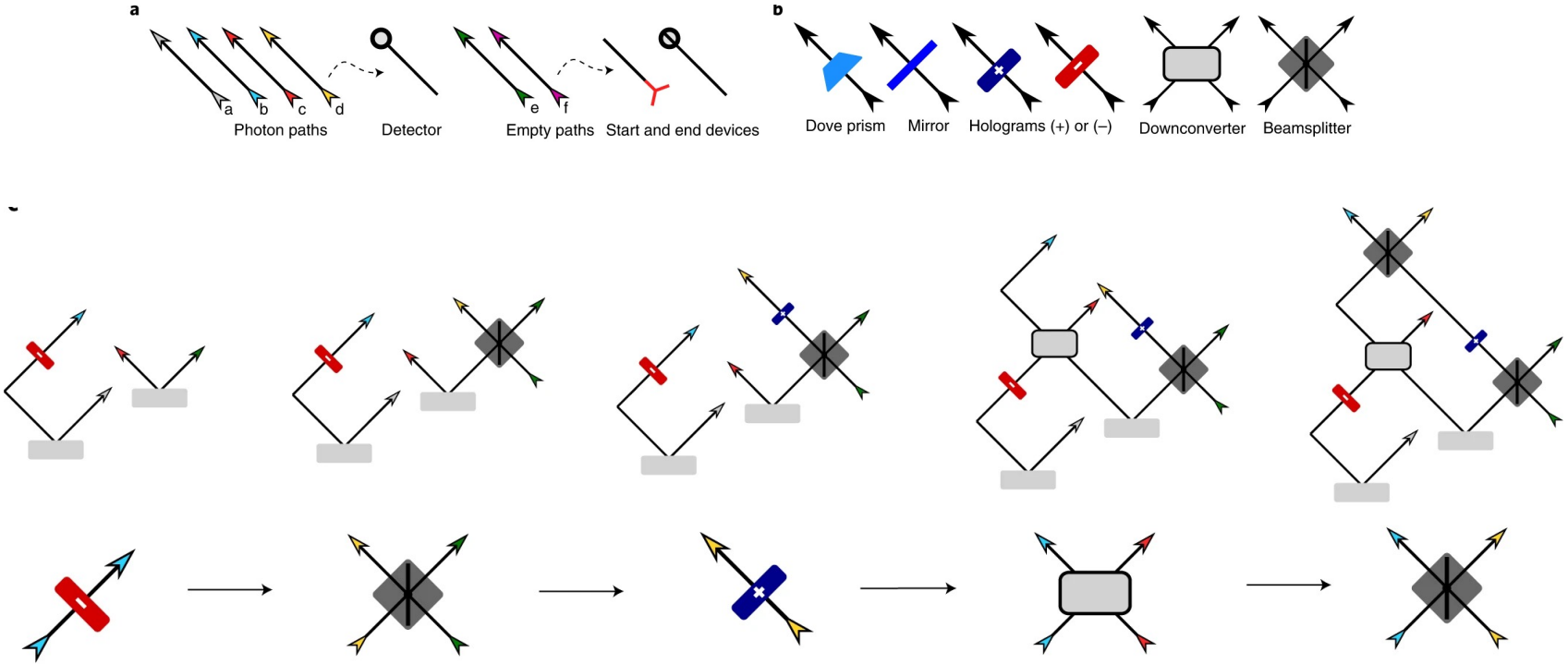
a) QO experimental setup

$$
\begin{aligned}
\mathrm{H}_a^1|\Psi\rangle &= |1\rangle_a \left(\tfrac{i|0\rangle_c+i|1\rangle_b}{\sqrt{2}}\right) + \left(\tfrac{|1\rangle_b-|0\rangle_c}{\sqrt{2}}\right)|0\rangle_d \\
&\quad +|0\rangle_c|0\rangle_d + i|1\rangle_c|1\rangle_d) + i|-1\rangle_c|-1\rangle_d \\
\mathrm{R}_c|\Psi\rangle &= |0\rangle_a \left(\tfrac{i|0\rangle_c+i|1\rangle_b}{\sqrt{2}}\right) + \left(\tfrac{|1\rangle_b-|0\rangle_c}{\sqrt{2}}\right)|0\rangle_d \\
&\quad +|0\rangle_c|0\rangle_d + i|1\rangle_c|1\rangle_d) + i|-1\rangle_c|-1\rangle_d \\
\mathrm{DC}_{cd}|\Psi\rangle &= |0\rangle_a \left(\tfrac{|0\rangle_c+i|1\rangle_b}{\sqrt{2}}\right) + \left(\tfrac{|1\rangle_b+i|0\rangle_c}{\sqrt{2}}\right)|0\rangle_d \\
&\quad +|0\rangle_c|0\rangle_d + |-1\rangle_c|1\rangle_d) + |1\rangle_c|-1\rangle_d \\
\mathrm{H}_b^1|\Psi\rangle &= |0\rangle_a \left(\tfrac{|0\rangle_c+i|1\rangle_b}{\sqrt{2}}\right) + \left(\tfrac{|1\rangle_b+i|0\rangle_c}{\sqrt{2}}\right)|0\rangle_d \\
\mathrm{BS}_{bc}|\Psi\rangle &= |0\rangle_a \left(\tfrac{|0\rangle_c+i|0\rangle_b}{\sqrt{2}}\right) + \left(\tfrac{|0\rangle_b+i|0\rangle_c}{\sqrt{2}}\right)|0\rangle_d \\
|\Psi\rangle &= |0\rangle_a|0\rangle_b + |0\rangle_c|0\rangle_d
\end{aligned}
$$

b) state calculation

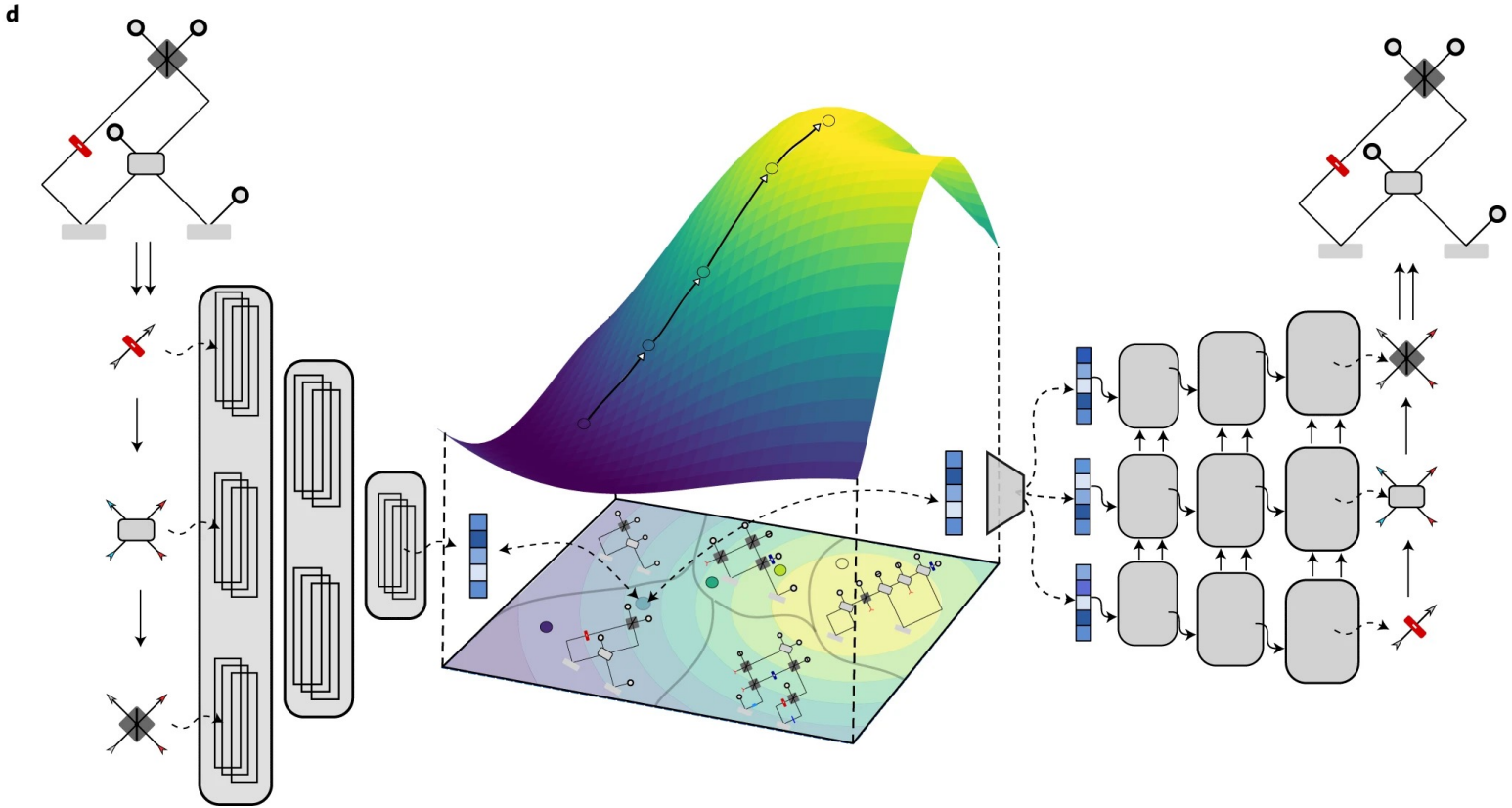
Learning interpretable representations of entanglement

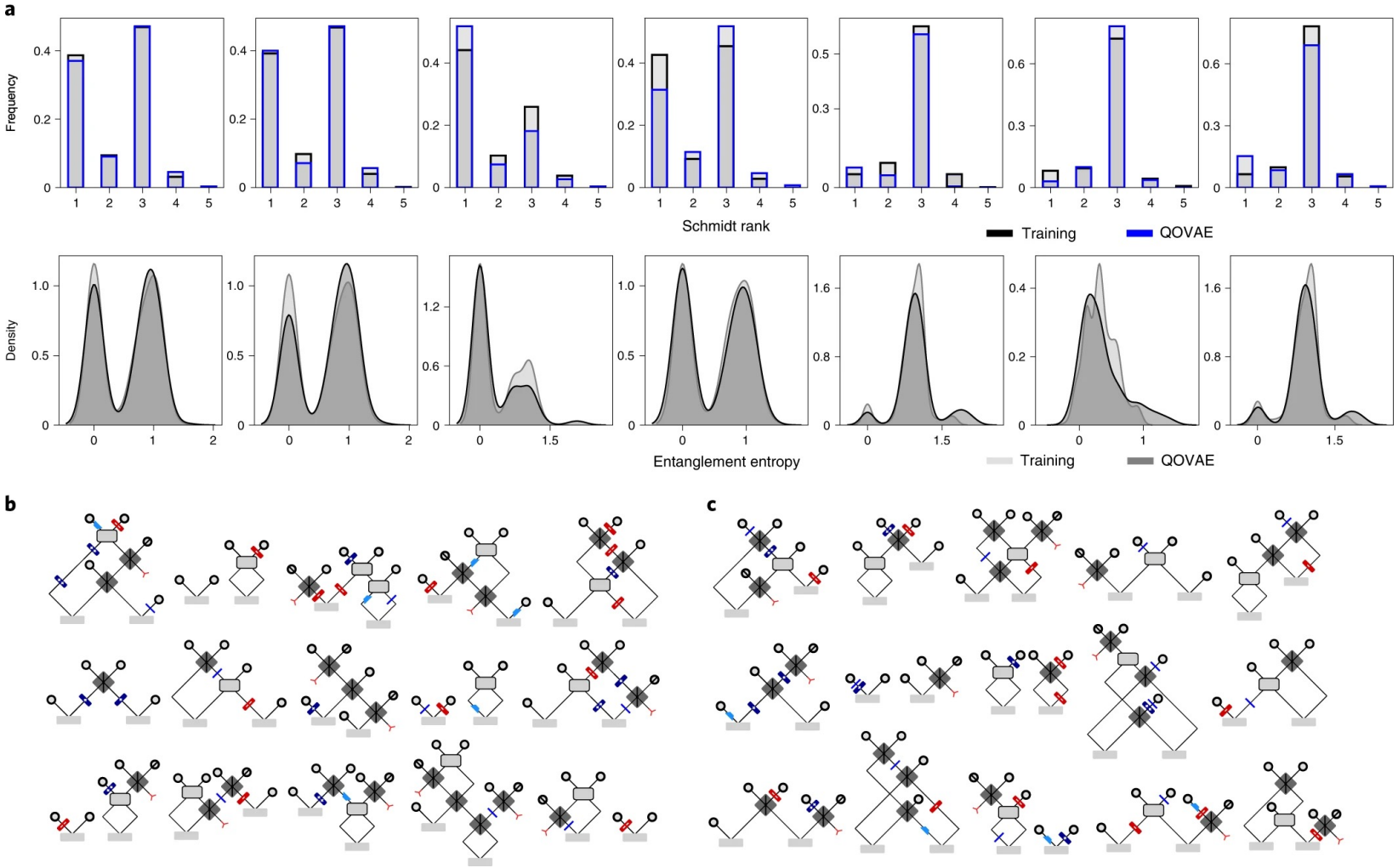
https://doi.org/10.1038/s42256-022-00493-5

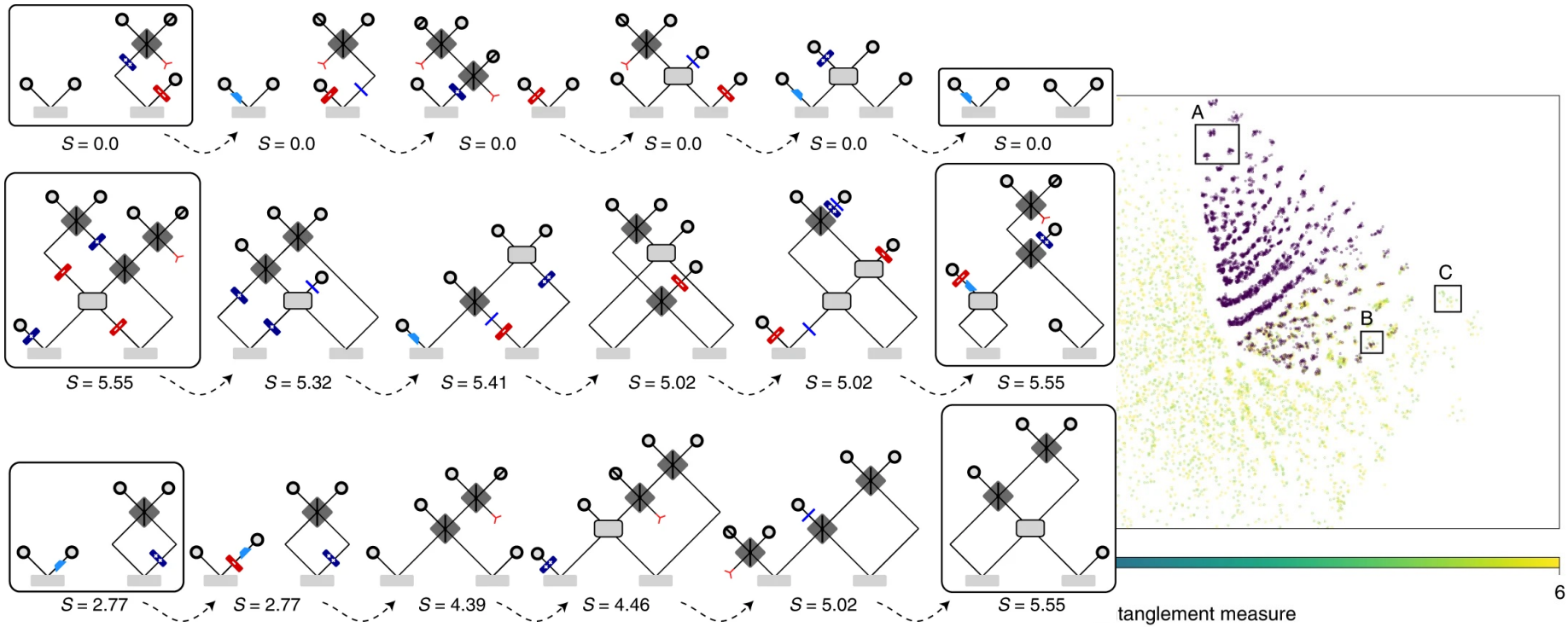


Learning interpretable representations of entanglement

https://doi.org/10.1038/s42256-022-00493-5

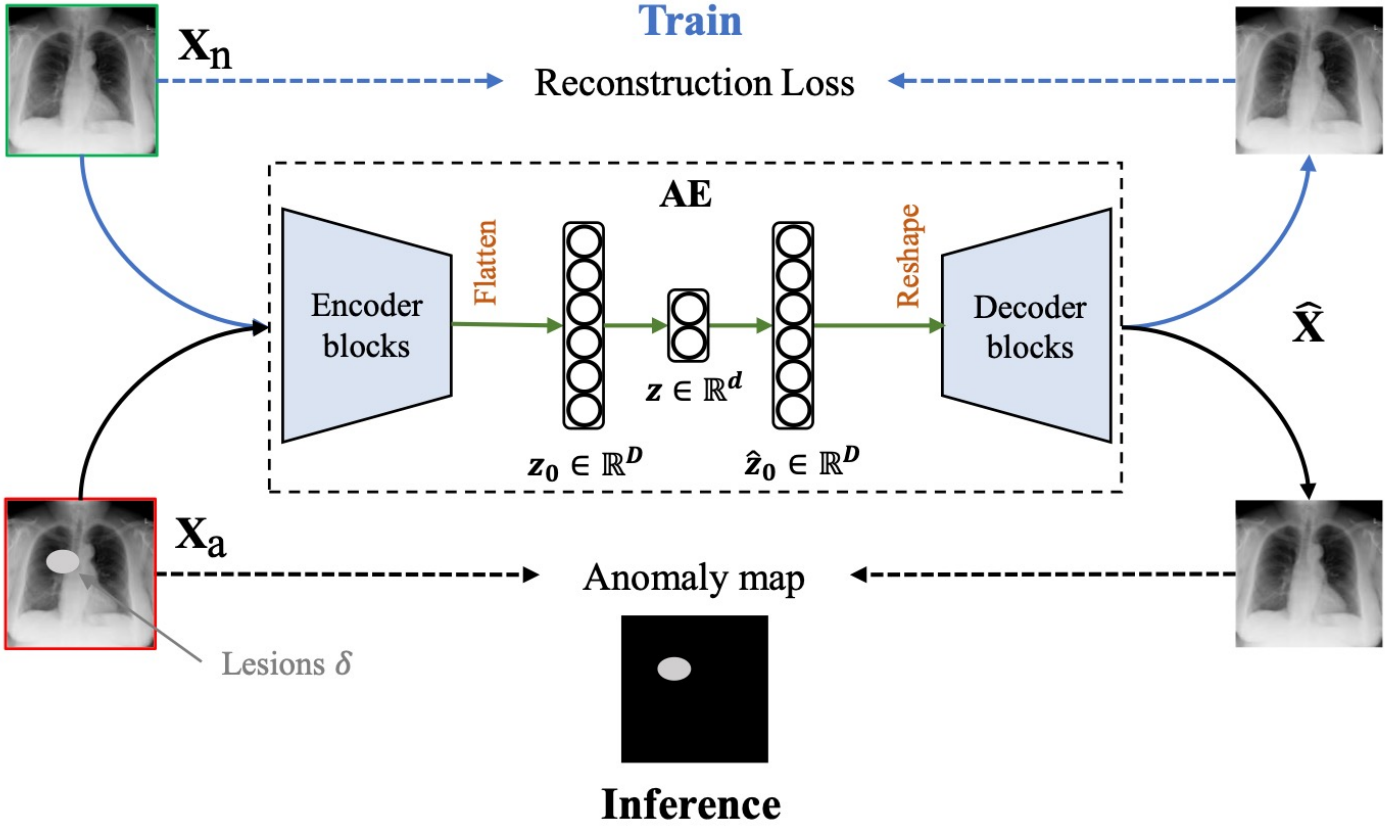



a
Frequency
Schmidt rank
Training
QOVAE
Density
Entanglement entropy
Training
QOVAE
b
c


S = 0.0
S = 0.0
S = 0.0
S = 0.0
S = 0.0
S = 0.0
S = 5.55
S = 5.32
S = 5.41
S = 5.02
S = 5.02
S = 5.55
S = 2.77
S = 2.77
S = 4.39
S = 4.46
S = 5.02
S = 5.55
A
B
C
6
tanglement measure

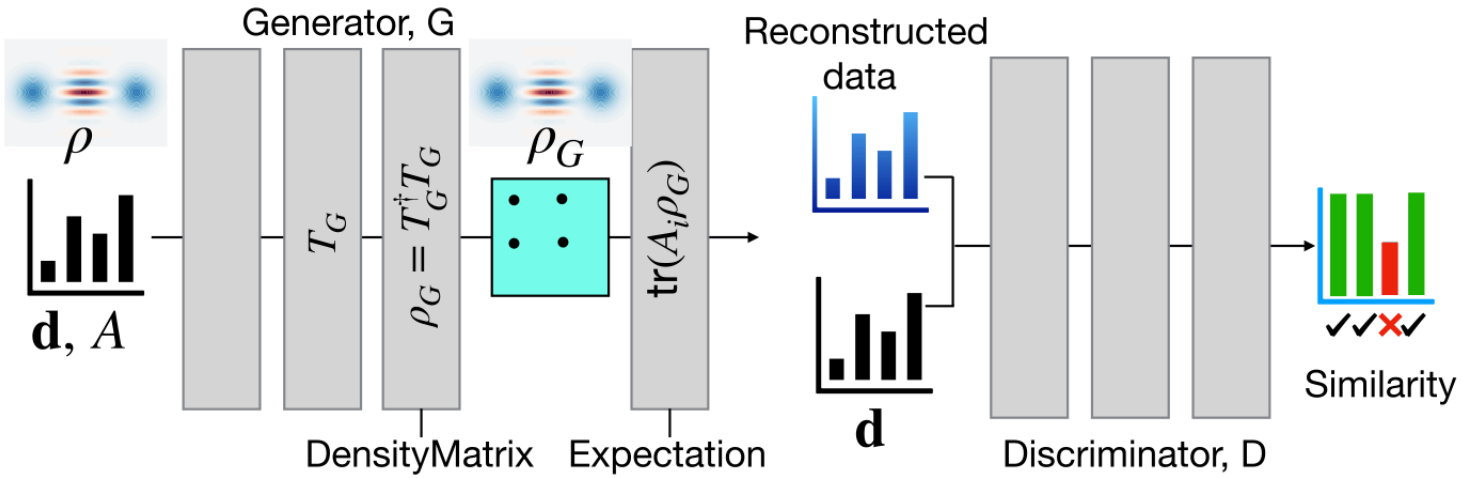
Anomaly Detection with Autoencoders

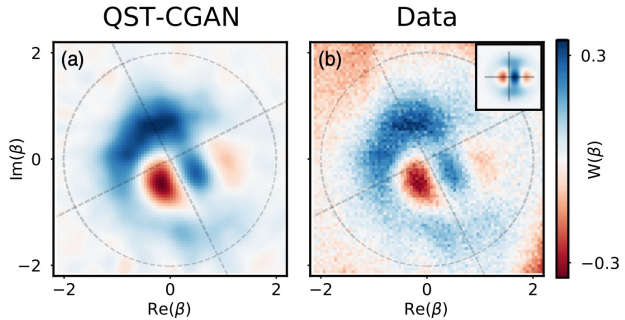
e.g., https://arxiv.org/abs/2403.09303



Quantum State Tomography with Conditional Generative Adversarial Networks

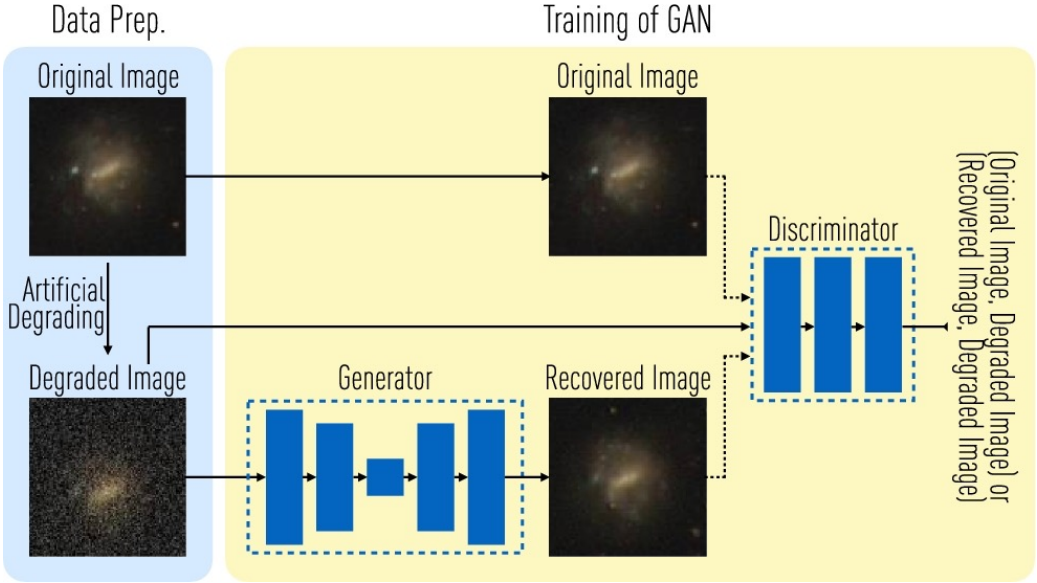
https://doi.org/10.1103/PhysRevLett.127.140502

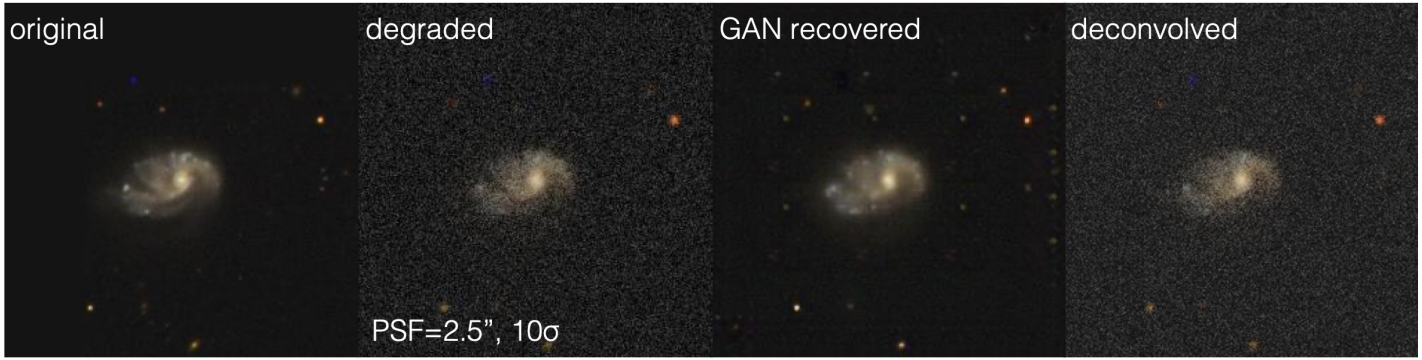




recover features in astrophysical images

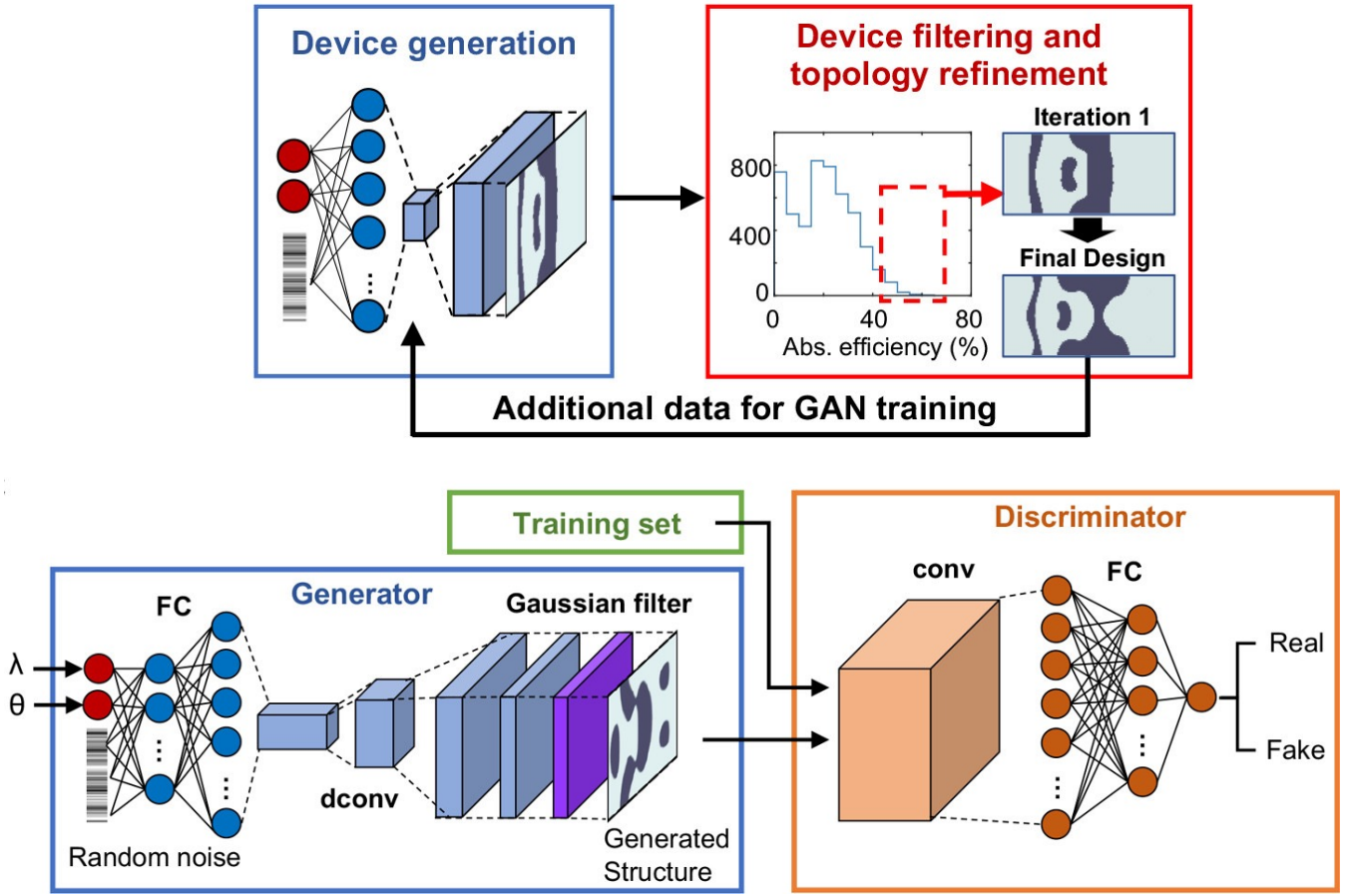
https://arxiv.org/abs/1702.00403



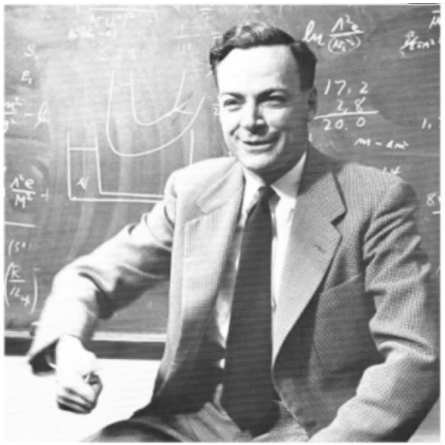


Diffractive Metagrating Design Based on Generative Adversarial Networks

https://doi.org/10.1021/acsnano.9b02371







Richard Feynman: “ What I cannot create, I do not understand”

Generative modeling: “ What I understand, I can **create**”